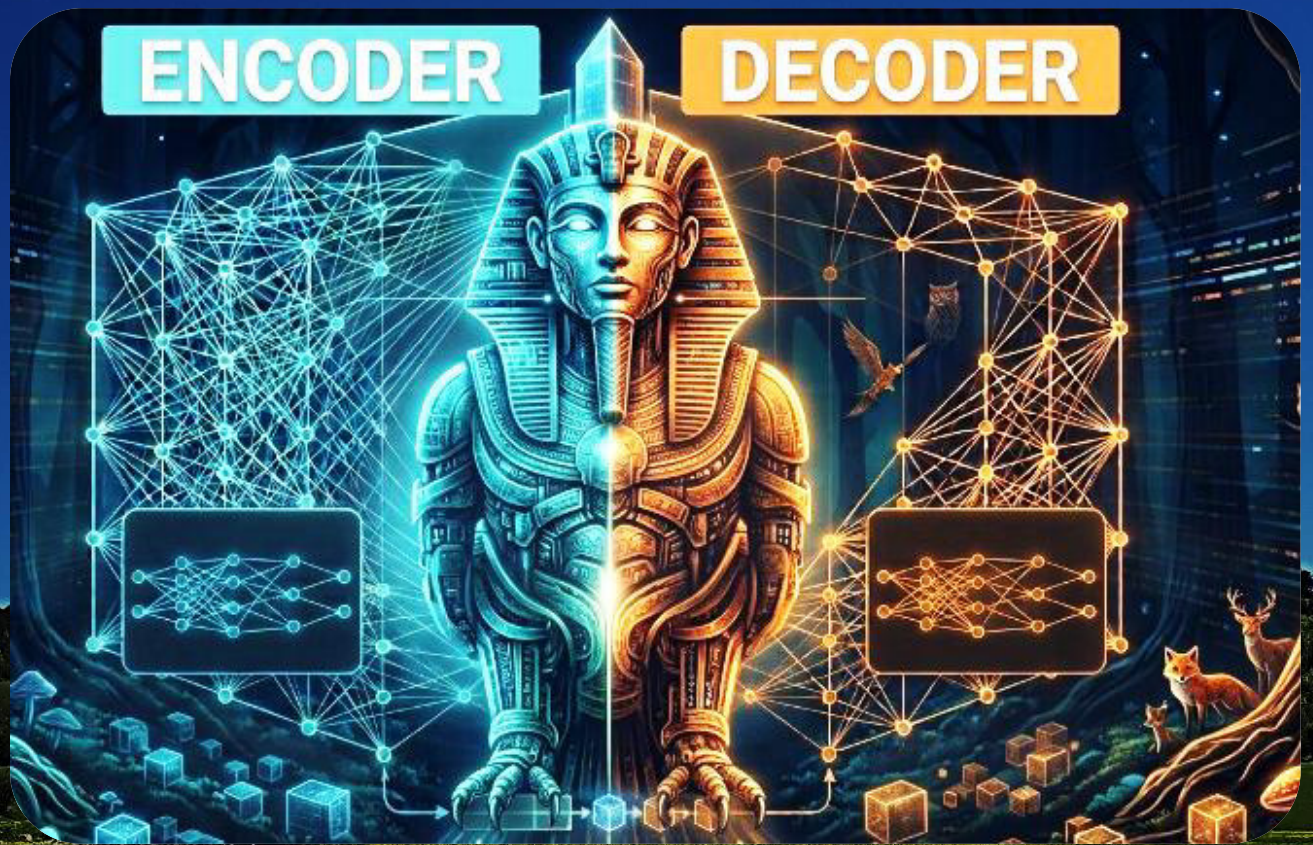


Inteligencia Artificial

Transformers



Yim APESTEGUI

Yim Isaias Apestegui Florentino

Inteligencia Artificial Transformers

Inteligencia artificial transformers

Autor – Editor:

© Yim Isaias Apestegui Florentino

Garatea Mz.100 Lote 7 – Nuevo Chimbote

yapestegui@gmail.com

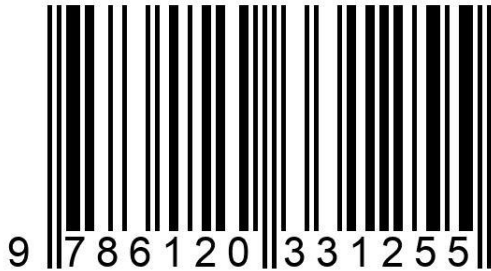
Ancash - Perú

Primera edición digital, setiembre 2026

Depósito Legal N° 2026-10658

ISBN:978-612-03-3125-5

ISBN: 978-612-03-3125-5



Libro disponible en: <https://yimaf.github.io/books/ia-transformers.html>

Tabla de contenido

Prólogo	vi
1 Introducción a la inteligencia artificial	1
2 Redes Neuronales	4
3 Introducción a los transformadores	8
3.1 ¿Qué son los transformadores?	8
3.2 ¿Qué es un Tokenizador?	10
3.3 Embedding.....	11
3.4 El problema del orden (la posición)	11
3.5 El ingreso al ENCODER	12
3.6 Self-Attention: cada palabra "mira" a las demás	13
3.7 Query, Key y Value	13
3.8 Cálculo de la atención	14
3.9 Multi Head Attention	16
3.10 Add	17
3.11 Norm.....	19
3.12 Feed-Forward Network (FFN).....	21
3.13 El objetivo del ENCONDER	22
3.14 Codificación de un ENCONDER	23
3.15 EL DECODER.....	48
3.16 Masked Self-Attention.....	49
3.17 Cross-Attention	53
3.18 Estructura muy completa de todo lo descrito	54
3.19 Como aprende el transformer.....	55

3.20	Código del DECODER para replicar	56
4	Epílogo.....	94
5	Bibliografía.....	95

Prólogo

Siempre me ha interesado como ingeniero la aplicación de los conocimientos y cómo la matemática ha sido y es la base de lo que hoy en día es la Inteligencia Artificial. Siempre menciono que gracias a la matemática la Inteligencia Artificial ha llegado a fronteras del conocimiento que no nos hubiéramos siquiera imaginado hace uno 5 años atrás.

En este libro, no quiero aburrirte con el hermoso arte de la matemática, aunque sí encontrarás pinceladas muy necesarias y fuertemente aplicada, porque aunque tu interés esté en el desarrollo de aplicaciones con IA, saber los fundamentos matemáticos que soportan la herramienta que utilizas te permitirá darle sentido a lo que haces, modificas o reescribes cuando codificas. Cada sección teórica está complementada con una aplicación que puedes reescribir, copiar o pegar en tu interfaz de desarrollo y verificar con tu propia experiencia los resultados mostrados.

Iniciamos con una introducción a la Inteligencia Artificial, que nos muestra un poco, pero muy poco de historia y más contenido respecto al estado del arte de la IA que llevó al desarrollo de Transformers. Luego hablamos de Transformers y así de literal “se descompone” al transformador para estudiar cada una de sus partes. Se presenta ejercicios de código que puedes ejecutar tanto para la fase ENCODER como la fase DECODER. Espero que este libro de ayude a comprender este interesante mundo de los Transformes en la IA.

1 Introducción a la inteligencia artificial

Hace unos años, nuestros profesores de inteligencia Artificial (IA), en la universidad, en la parte introductoria de la asignatura de IA, nos narraban de Turing y su famosa prueba “El Test de Turing”. Y veíamos como todo parecía ciencia ficción y decíamos: ¿cuándo será el día que no se distinga si algo realizó una máquina o un ser humano?. Hoy eso, es cosa del pasado, efectivamente de los años 50.

En 2026 la IA ya no es ciencia ficción; es la herramienta que más se utiliza en universidades, colegios, negocios, etc. Por supuesto que los negocios son quienes más está interactuando con la IA, optimizando costos e indicadores del aspecto laboral (Lightcast, s. f.). Diríamos así: la historia va desde los “chatbots” o la “IA conversacional” hacia sistemas capaces de planear y ejecutar tareas por sí solos; la “IA Agéntica” (Standford, 2026).

Si hablamos de sistemas capaces, ni se diga. Ya a estas alturas nos damos cuenta que la IA en ciertas actividades de procesamiento nos lleva años luz, pues su velocidad de procesamiento es increíble. Por ejemplo en el examen "Humanity's Last Exam", que es un examen a prueba de “TODO” para ser resueltas por la IA, con preguntas elaboradas por especialistas de altísimo nivel de todo el planeta (HLE, 2026). El mejor modelo de 2025 llegaba al 8,8% de aciertos, en abril de 2026, modelos como Claude Opus 4.6 de Anthropic o Gemini 3.1 Pro de Google bordean más del 50%. Aunque los investigadores del índice de IA de Stanford advierten, que los resultados en un benchmarking no garantizan que el sistema funcione igual de bien en un contexto real y específico, al menos, digo yo, ya nos da un horizonte de hacia dónde apunta el “razonamiento” de la IA. No entraré en discusiones filosóficas de que si la IA razona o no razona, por ahora, por eso las comillas de “razonamiento”. En uno de los métodos que las herramientas de IA

utilizan, los autores le pusieron “fase de razonamiento” y en este contexto diremos que la IA al menos tiene una fase de “thinking”.

En una encuesta a más de diez mil líderes organizacionales se encontró que la mayoría todavía piensa que en la IA como herramienta de apoyo para los próximos dos años, y no como un reemplazo de personas o trabajadores, aunque los que estamos en el área de desarrollo de sistemas, sabemos qué actividades ahora lo realiza la IA y ya no nosotros. De hecho es preocupante por un lado, pero también desafiante y más aún cuando estamos en un entorno y una infraestructura heredada que no está lista para estos sistemas (McKinsey, 2026).

No debemos dejar de mencionar que detrás de la IA hay una inmensa cantidad de dinero, tal vez ni podamos imaginarnos. Te daré algunos datos: Google reportó más de \$150.000 millones como gasto de capital en 2025, invertidos en infraestructura de cómputo para IA, y el gasto total se ha multiplicado por treinta desde 2021. Nvidia, “la protagonista” controlando más del 60% de la capacidad instalada en todo el mundo. La gente común empieza a sentir el excedente que los consumidores estadounidenses, por ejemplo, obtienen de estas herramientas pues casi siempre gratuitas o muy baratas pasó de 112.000 a 172.000 millones de dólares anuales en tan solo un año, apreciado lector (Standford, 2026).

En la Unión Europea, la Ley de IA ya entro en vigor y rigor; desde febrero de 2025 están prohibidos los sistemas de “riesgo inaceptable”, desde agosto de 2025 se debe cumplir con las obligaciones de transparencia y documentación técnica y desde el 2 de agosto de 2026 (un mes antes que publique este libro) entra en vigencia el artículo 50 y su reglamentación. Algunas exigencias “de mayor riesgo” se han aplazado hasta diciembre de 2027 (Trova, 2026).

Como verás apreciado lector, esta introducción a la IA, no ha empezado con la historia de la IA como tal, que de hecho me gustaría que la

supieras, porque he querido aportarte información más reciente, para ponernos en contexto. De hecho, te recomiendo el Libro de Rousell y Norving en su introducción podrás leer el pasado de la IA y el estado del arte de aquellos años, es un libro demasiado bueno, sólido y profundo (Russell & Norvig, 2011).

Con todo lo anteriormente mencionado: modelos que aprueban exámenes diseñados para y por expertos, empresas que invierten millonadas de dólares, agentes que ya están automatizando algunos trabajos y elementos legales y corriendo detrás de todo eso, talvez no tenga mucho sentido una "introducción a la inteligencia artificial" que nos explique qué es una red neuronal desde cero. Sino más bien, entender un ecosistema de sistemas IA y no IA en plena fricción entre lo que la tecnología ya puede hacer, y lo que la sociedad todavía está negociando sobre cómo debería usarse, muy natural en las Revoluciones Industriales de la historia del mundo. Lo real es que para bien o para mal, la IA sigue operando en 2026 y estimo que con mayor auge en los años que vengan.

2 Redes Neuronales

En la sección anterior, mencioné que tal vez no tenía mucho sentido, explicar en esa sección, qué es o cómo es una red neuronal desde cero, pues hoy la IA se encuentra usando arquitecturas más complejas de redes neuronales. Sin embargo, en esta sección abordaremos dicho tema, no tanto desde cero pero sí la parte fundamental de las redes neuronales.

En 1943, el neurofisiólogo Warren McCulloch y el matemático lógico Walter Pitts querían emular al cerebro pensando que se podía representar como un circuito lógico de interruptores binarios. Presentaron un modelo matemático llamado neurona que "dispara" o no dispara dependiendo que si la suma de sus entradas supera un umbral, y con eso demostraron que una neurona en teoría calcularía cualquier función lógica (McCulloch & Pitts, 1943). En otras palabras dicha neurona es el génesis de todo lo que hoy llamamos IA basada en redes neuronales.

Quince años después aproximadamente, Frank Rosenblatt le agregó un concepto alucinante, que es la capacidad de aprender, así es como puedes leerlo. Nace el Perceptrón ya no como un simple modelo matemático, sino como una máquina capaz de autoajustar sus propios pesos internos, corrigiendo el error cada vez que no acertaba en una clasificación (Rosenblatt, 1958).

La alegría no duró mucho porque en 1969 Minsky y Papert publicaron artículo donde mediante un análisis matemático mencionan que un perceptrón de una sola capa no podía resolver problemas tan simples como la función XOR.

A continuación un poco de teoría y práctica respecto a XOR. No quiero ofenderte, pero recuerda que este texto es para programadores junior, por eso mencionamos cosas básicas pero fundamentales. Básicamente si los valores de verdad a y b difieren, el resultado es *verdadero* y si los valores de verdad a y b son iguales el resultado es *falso*.

Entrada a	Entrada b	$a \text{ XOR } b$
0	0	0
0	1	1
1	0	1
1	1	0

En el código Python siguiente, la función `fxor` nos permite verificar cómo funciona la función XOR. Además debo indicarte que está que Python tiene librerías que permiten usar la función XOR, como la librería `numpy`, `scipy`, `scikit-learn`, `tensorflow` o `pytorch`.

pruebaXoR.py	
<pre>def fxor(a, b): # Devuelve True si son diferentes, False si son iguales return bool(a) != bool(b) # Pruebas print(fxor(False, False)) # Resultado: False print(fxor(False, True)) # Resultado: True print(fxor(True, False)) # Resultado: True print(fxor(True, True)) # Resultado: False print(fxor(0, 0)) # Resultado: False print(fxor(0, 1)) # Resultado: True print(fxor(1, 0)) # Resultado: True print(fxor(1, 1)) # Resultado: False</pre>	

Salida:

```
False
True
True
False
False
True
True
False
```

Y es así como el pobre y triste Perceptrón quedó frenado durante casi una década, algunos autores lo llaman a esta época como la época del "primer invierno de la IA" (Minsky et al., 2017).

En 1986, se publica el algoritmo de retropropagación del error muy conocido en nuestra área como backpropagation, donde se resuelve el problema que congeló a la IA. Y esta solución ahora sí permitía entrenar redes con varias capas ocultas, propagando el error desde la salida hacia atrás con el objetivo de ajustar cada peso proporcionalmente al fallo final (Rumelhart et al., 1986). Estimado lector, ya han pasado más de 40 años y este fundamento (backpropagation) aún sigue siendo, obviamente con variantes y optimizaciones, el punto central de casi todo el entrenamiento de arquitecturas de redes neuronales modernas.

Avanzado en nuestra interpretación: una red neuronal es una serie de capas de neuronas, las cuales reciben números, los multiplica por pesos, le suma un sesgo, y pasa el resultado para ser evaluada por una función no lineal (puede ser ReLU o sigmoide, etc.) antes de pasar a la siguiente capa. ¿Vas comprendiendo?.

Entonces, el aprendizaje consiste en ir ajustando los pesos, en realidad pueden llegar a ser miles o millones de ellos, dado un conjunto de ejemplos de entrada y salida deseada o esperada. Así surge, el proceso en que el error que encuentra la red neuronal y lo correcto se reduzca poco a poco. Ese proceso de ajuste se hace con un fundamento matemático denominado descenso del gradiente, y backpropagation se convierte simplemente en el método eficiente para calcular y determinar cómo debe moverse cada peso.

A partir de entonces los tipos de redes neuronales empezaron a ramificarse. Surgen las redes neuronales convolucionales (CNN), de Yann LeCun que reconocen dígitos escritos a mano, usan filtros que se deslizan sobre la imagen y se empieza a detectar patrones locales como bordes, texturas, formas, tamaño y luego los van combinando hacia capas más profundas y más profundas (literal) hasta reconocer objetos completos (Krizhevsky et al., 2017).

Apreciado “junior”, ahora sí, se dio el “gran salto” que las puso de moda en aquel entonces, como lo es hoy la “IA agentica”: nada más y nada menos que AlexNet, una CNN entrenada con unidades gráficas de procesamiento (GPUs), quien ganó la competencia ImageNet en 2012, y se dio el momento en que básicamente nace la moda del aprendizaje profundo “deep learning” que sinceramente veo que sigue siendo uno de los estados del arte de la investigación en estos tiempos y te hablo de 2026.

Ahora vamos por otro linaje de “pura sangre” me refiero a las redes neuronales recurrentes (RNN), diseñadas para trabajar datos secuenciales como texto, audio o series temporales, así como el ser humano trabaja de manera secuencial, porque mantienen algo así como una “memoria interna” y que se actualiza en cada paso o proceso de la secuencia.

El problema de estas famosas RNN es lo que sucede cuando las secuencias son larguísimas, te lo digo en palabras comunes: la memoria muere, Lo que sucede es que hay degradación de memoria y surge otro acontecimiento matemático (el famosísimo problema del desvanecimiento del gradiente). Los investigadores proponen una solución decorosa la “Long Short-Term Memory” (LSTM), que introduce un mecanismo para decidir qué información se debe conservar y qué información se lleva al olvido (Hochreiter & Schmidhuber, 1997).

Debo mencionarte que existen redes generativas antagónicas (GAN): Una que genera datos falsos y otra que intenta distinguirlos de los reales. Por ejemplo generadores capaces de crear imágenes muy realistas (Goodfellow et al., 2014). Y los autoencoders, que comprimen una entrada en una representación reducida para luego reconstruirla usado en detección de anomalías y son la base de modelos generativos más recientes (Hinton & Salakhutdinov, 2006).

Exactamente en 2017 surgen los “Transformers”, un equipo de Google dijo: No a la recurrencia y además el procesamiento de secuencias se debe hacer

con mecanismo de "atención". Este mecanismo le permite al modelo ver todas las palabras de un texto u oración, al mismo tiempo. Además permite ponderar cuáles son relevantes para entender a cada una de las otras palabras (Vaswani et al., 2023).

Recomiendo que leas el artículo de Vaswani, dado que constituye la base arquitectónica de los modelos modernos de lenguaje de hoy en día: DeepSeek, Claude, ChatGPT, Copilot, etc.

3 Introducción a los transformadores

3.1 ¿Qué son los transformadores?

Trataré de conceptualizar la idea de transformadores en términos que sean entendibles por personas que por primera vez escuchan esta palabra. De hecho me estoy dirigiendo al programador junior a aquel que por alguna razón o de alguna manera o simple curiosidad ha llegado hasta aquí.

En un inicio se usaban redes neuronales recurrentes (RNN) o redes neuronales convolucionales (CNN) para tareas como traducir textos, por ejemplo complementados con un mecanismo denominado mecanismo de atención. En 2017 aparece un artículo denominado "Attention Is All You Need" donde explica dicho mecanismo, cabe mencionar que este artículo ha sido actualizado hasta el 2023 (Vaswani et al., 2023).

Un transformador no es otra cosa que un modelo un tanto complejo muy parecido a las redes neuronales pero que deja a un lado las RNN y las CNN a fin de quedarse solo con el mecanismo de atención y hacer procesamiento en paralelo más rápido o más eficiente, cosa que con las redes anteriores hacer paralelismo tenía obstáculos debido a la secuencialidad (no paralelismo) de las redes neuronales artificiales (RNA).

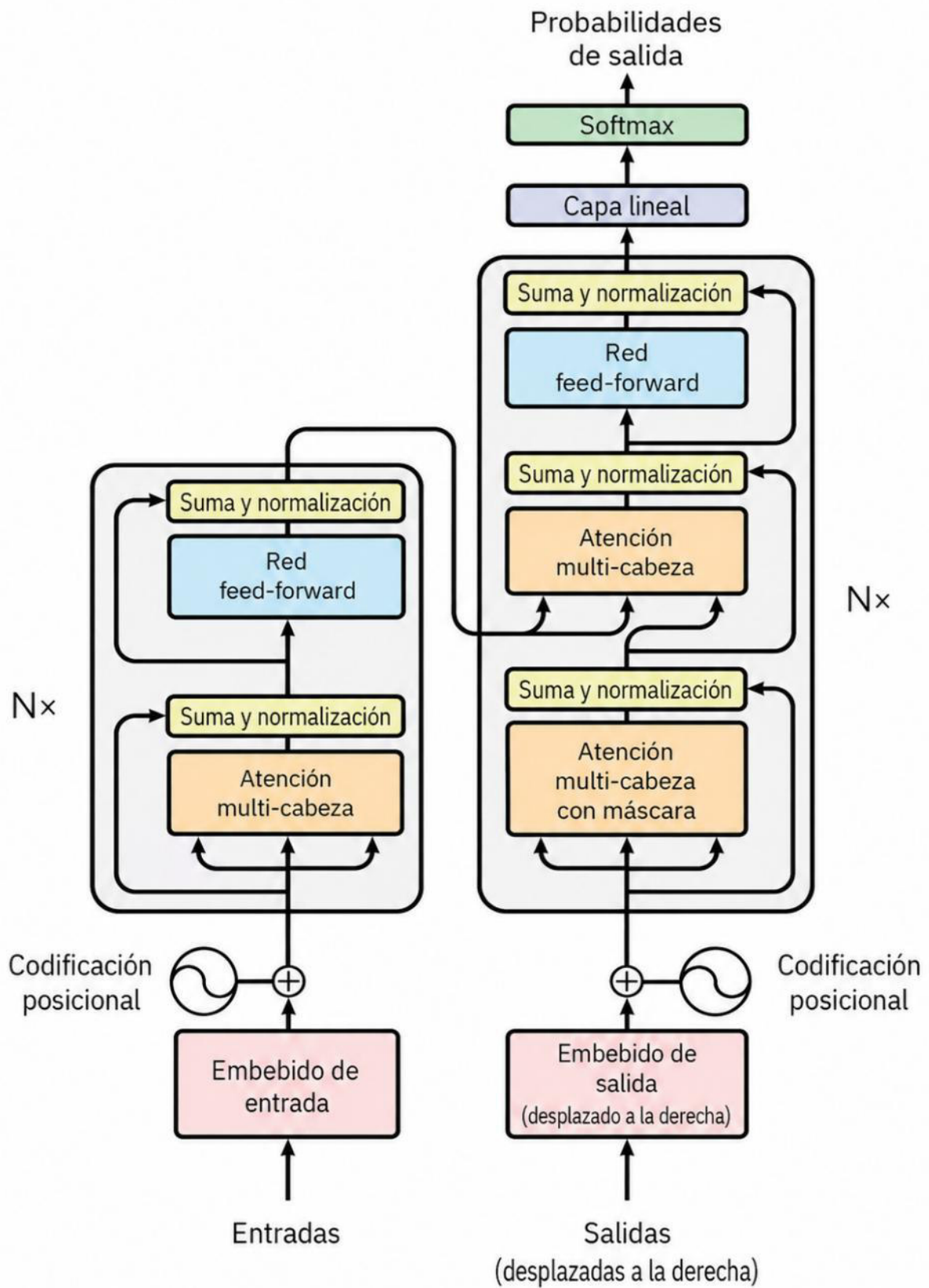


Fig. 1 Imagen de la arquitectura de un Transformers adaptado de (Vaswani et al., 2023).

En las secciones anteriores ya he mencionado mi querido lector, que la idea fundamental es diferente a la de una red recurrente (RNN/LSTM). ¿Por qué?, porque no se necesita procesar secuencialmente todos los elementos uno por uno para comprender la entrada. Aquí es donde se utiliza el mecanismo self-attention, que hace que cada elemento de la secuencia determine qué otros elementos son realmente relevantes para interpretarlo.

Veamos un ejemplo:

Tenemos la oración: **“Se acerca el fenómeno El Niño y las casas de la costa no están preparadas para lluvias intensas.”**

Como programador podrías pensar crear un variable y asignar todo el texto a una variable:

`texto = "Se acerca el fenómeno El Niño y las casas de la costa no están preparadas para lluvias intensas."`

Pero el Transformers no recibe texto directamente, hay que convertirlo en **tokens**.

¿Tokens?, -si junior tranquilo no te preocupes- **tokens**.

3.2 ¿Qué es un Tokenizador?

Un tokenizador, divide a las oraciones en unidades que el modelo ya conoce, para el caso del ejemplo sería más o menos así:

<code>[Se] [acerca] [el] [fenómeno] [El]</code> <code>[Niño] [y]</code> <code>[las] [casas] [de] [la] [costa] [no]</code> <code>[están]</code> <code>[preparadas] [para] [lluvias] [intensas]</code>	<code>Donde n=18</code> <code>tokens</code>
--	--

Por ahora 18 tokens, aunque en un modelo más sofisticado, las palabras podrían segmentarse aún más, como:

`preparadas → prepara + das`

De otra parte, se puede representar los tokens con sus respectivos Ids

Se -> 1523

```

acerca    -> 8742
el        -> 35
fenómeno  -> 9182
El        -> 127
Niño      -> 5631
...

```

Python
tokens = [1523, 8742, 35, 9182, 127, 5631, ...]

3.3 Embedding

Ahora cada ID se transformará en un vector. Supongamos que nuestro modelo utiliza vectores de solo 6 dimensiones. Aunque en un modelo más aplicado a la real podrían ser cientos o miles de dimensiones.

```

Se        -> [ 0.21, -0.34, 0.72, 0.15, -0.11, 0.43]
acerca    -> [ 0.61, 0.12, -0.28, 0.81, 0.32, -0.07]
el        -> [ 0.14, 0.08, 0.21, -0.11, 0.05, 0.18]
fenómeno  -> [ 0.77, -0.23, 0.51, 0.63, 0.42, -0.31]
El        -> [...]
Niño      -> [...]
...

```

Podríamos entonces generalizar que nuestra oración es una matriz de X elementos donde:

$$X \in R^{18 \times d_{model}}$$

En una relación del orden den 18 tokens x d_{model} . El número de dimensiones de d_{model} es de 6 dimensiones en nuestro caso. Pero también puede ser de 512, 1024, 2048, etc. dimensiones.

3.4 El problema del orden (la posición)

Todo iba bien hasta aquí, pero trabajar solo con embeddings no es suficiente; porque el modelo ¿cómo podría distinguir?

“El Niño se acerca” de “Se acerca El niño”

Entonces, necesitamos información de “la posición”:

Posición	Token
0	Se
1	acerca
2	el

3	fenómeno
4	El
5	Niño
6	y
7	las
8	casas
9	de
10	la
11	costa
12	no
13	están
14	preparadas
15	para
16	lluvias
17	intensas

=====

Podemos generalizar esta parte de la siguiente manera:

$$X_i = \text{Embedding}(\text{token}_i) + \text{Posición}(i)$$

DE esta manera tenemos el elemento X, conformado por su embendings más su posición.

$$\text{Embedding}(\text{"Niño"}) + \text{Posición}(5) = \text{Representación inicial de "Niño"}$$

Solucionado el problema, porque nuestro “modelito” ya sabe qué palabra es y en qué posición u orden aparece ¡Genial!.

3.5 El ingreso al ENCODER

Es el momento de ingresar al ENCODER de la arquitectura y es aquí donde la cosa se torna más interesante.

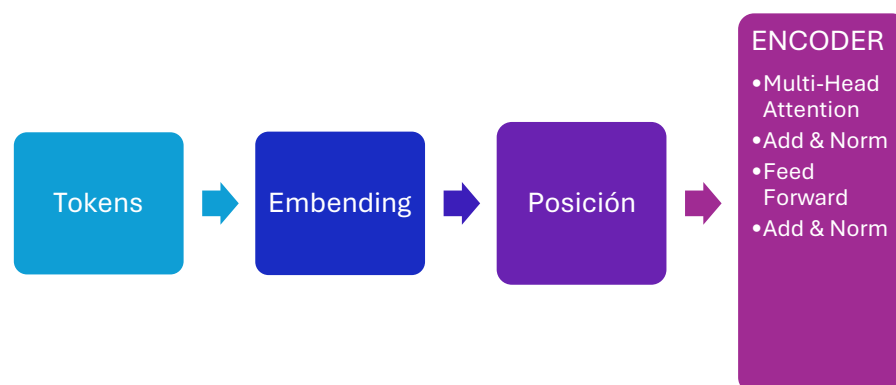


Fig. 2 Ingreso del Token al ENCODER

3.6 Self-Attention: cada palabra "mira" a las demás

Consideremos específicamente: la palabra “Niño”. Nuestro modelo necesita determinar ¿qué significa Niño? dentro de esta oración. Si solamente miramos la palabra Niño, podríamos suponer o pensar que:

un niño = una persona

Pero en realidad el contexto es otro: **fenómeno El Niño** y además tenemos:

Se acerca el fenómeno El Niño

Por tanto, Niño debe relacionarse fuertemente con:

Fenómeno, El, acerca

y posteriormente con:

lluvias, costa

Self-attention permite construir precisamente esas relaciones, ¿Cómo lo hace?. Aquí es donde aparece, Query, Key y Value.

3.7 Query, Key y Value

En los modelos Transformer Query (Consulta), Key (Clave) y Value (Valor) son tres vectores del mecanismo de self_attention, funcionando de manera parecida a un sistema de búsqueda en una base de datos. A partir de la matriz de representación X, el Transformer calcula:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

Python: @ representa es un operador de multiplicación matricial
Q = X @ W_Q
K = X @ W_K
V = X @ W_V

Volviendo a la palabra “Niño”, ¿Qué está preguntando “Niño”? Lo que sucede es que cuando el token Niño genera su Query, conceptualmente está buscando:

"¿Qué otros tokens de esta oración son relevantes para determinar mi significado?"

Las Keys de los demás tokens responden, parafraseando los resultados matemáticos:

fenómeno : "yo puedo estar relacionado contigo"

costa : "yo tengo cierta relación"

lluvias : "yo tengo relación"

casas : "yo tengo relación"

programador : "no estoy relacionado"

Esto no ocurre mediante reglas programadas o definidas explícitamente en el código, sino que es el resultado de que la red neuronal aprendió esas relaciones durante el entrenamiento.

3.8 Cálculo de la atención

La fórmula de la atención es la siguiente:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Donde:

QK^T : calcula qué tan relacionadas están las consultas con las claves

Supongamos que en un escenario, hubiéramos obtenido los siguientes datos:

Relación de "Niño" con:

=====	
Se	0.02
acerca	0.08
el	0.05
fenómeno	0.31
El	0.24
Niño	0.10
y	0.01
las	0.02

casas	0.03
de	0.01
la	0.01
costa	0.04
no	0.01
están	0.01
preparadas	0.01
para	0.01
lluvias	0.03
intensas	0.02

Aunque los datos son datos son ilustrativos. Es importante observar que **Niño** prestaría más atención **fenómeno, El;** que a palabras como **las, de, para.**

Entonces, luego de aplicar “atención”, *Niño* deja de estar representado simplemente como *Niño*. Ahora tenemos a un:

Niño + contexto

Es decir:

Niño en el contexto de fenómeno

O sea, el Transformer no cambia el significado de la palabra sino construye una representación contextualizada. Si vemos la palabra "costa", la palabra aparece en:

las casas de la costa, pero el contexto completo contiene:

El Niño

lluvias intensas

casas

costa

preparadas

Por tanto, el modelo puede aprender relaciones como:

costa ↔ casas

costa ↔ lluvias

costa ↔ El Niño

costa ↔ preparadas

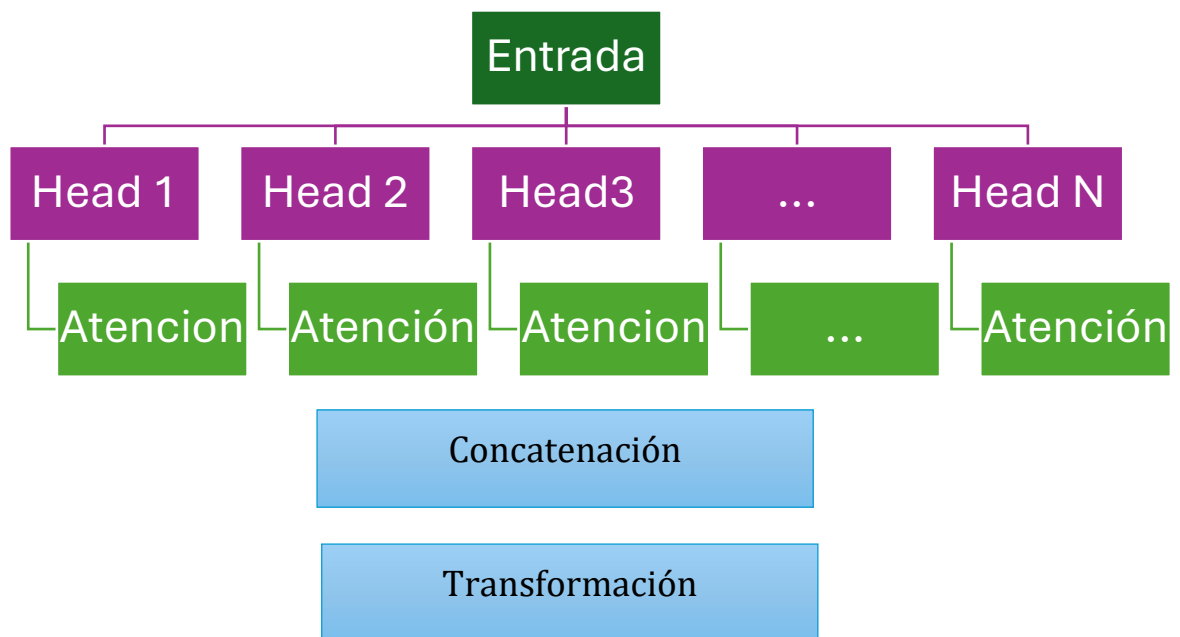
Una representación podría ser:

costa + casas + lluvias + El Niño + preparadas

Como se puede apreciar a este nivel, la representación que sale del Transformer contiene mucho más contexto y es más expresivo que el embedding original.

3.9 Multi Head Attention

En realidad no existe una única atención sino varias atenciones, por ello la denominación Atención Multi Cabeza, donde cada cabeza aprende diferentes patrones.



Por ejemplo, tenemos el siguiente caso de Head Attention

Head 1

"fenómeno" ↔ "Niño"

Head 2

"casas" ↔ "costa"

Head 3

"lluvias" ↔ "intensas"

Head 4

"casas" ↔ "preparadas"

Head 5

"Niño" ↔ "lluvias"

No debemos inferir de manera literal que el modelo asigna una función fija a cada cabeza. Lo que pasa, estimado lector, es que cabezas aprenden diferentes patrones de atención a partir de los datos.

Inicialmente teníamos:

Se acerca el fenómeno El Niño ...

Después de la atención, cada token tiene una representación que incorpora información (contexto) del resto de la oración.

"Niño" ↔ "fenómeno El Niño"

"lluvias" ↔ "lluvias asociadas al contexto climático"

"casas" ↔ "casas ubicadas en la costa"

"preparadas" ↔ "casas que no están preparadas"

La representación de un token deja de depender únicamente del token y comienza a depender del contexto.

3.10 Add

El nombre Add & Norm viene Add: sumar la entrada original con la salida de la subcapa, y Norm: normalizar el resultado.

Cuya fórmula matemática es la siguiente:

$$Y = \text{LayerNorm}(X + \text{Attention}(X))$$

Donde:

LayerNorm, puede ser Multi Head Attention o Feed Forward Network

A ver vamos paso a paso, consideremos la palabra "casas"

Después de haber calculado el embedding y de algunas “transformaciones se tiene: $X=[0.2,0.5,-0.1,0.8]$, luego de pasar por Multi-Head Attention se obtiene: $Attention(X)=[0.4,-0.2,0.3,0.6]$. De acuerdo a la fórmula de Add:

$$X + Attention(X)$$

$$[0.2, 0.5, -0.1, 0.8] + [0.4, -0.2, 0.3, 0.6] = [0.6, 0.3, 0.2, 1.4]$$

Te das cuenta, no se trata solo de quedarnos con el dato de $Attention(X)$, sino de conservar la información que ya tenía X .

Técnicamente se conoce como conexiones residuales o también como skip connection. Duchi de otra manera, la información inicial u original no pasa obligatoriamente por la transformación puede hacer un skip connection.

Imagina que tienes un Transformers con 6 o 12 o 48 o más capas la información tendría que atravesar por muchas capas es decir:

$$X, Layer1, Layer 2, Layer 3, ..., Layer N$$

Cuando estás en una fase de entrenamiento, recuerda que los gradientes y sus problemas asociados empiezan a aparecer y se vuelven complicados para transmitirse entre tantas capas.

En cambio con conexiones residuales, es otra cosa. Cada bloque aprende principalmente si es que necesita hacer modificación alguna sobre la representación existente, en lugar de reconstruir la representación desde un inicio.

Sea X = la información que ya se tiene y

$Attention(X)$ = nueva información descubierta, entonces:

$$X + Attention(X)$$

Te lo diré así:

“Lo que ya se sabe o se tiene información, consérvalo. Y añade información de contexto que ha aportado la atención.”

Te hago recordar nuestro texto:

"Se acerca el fenómeno El Niño y las casas de la costa no están preparadas para lluvias intensas."

Para el texto, antes de atención “casas” es la representación inicial, después de la atención se incorpora, se añade “costa”, “lluvias”, “El Niño”, “preparadas”. Ves?. La conexión residual conserva la información inicial mientras se incorpora información de contexto.

3.11 Norm

Des pues de la suma, que se ha descrito en la sección anterior ahora estudiaremos LayerNorm, es decir: $Y = \text{LayerNorm}(X + \text{Sublayer}(X))$ que tiene similitud con :

$$Y = \text{LayerNorm}(X + \text{Attention}(X))$$

La normalización ayuda a conducir las activaciones en escalas más fáciles de manejar y además favorece al entrenamiento dándole mejor estabilidad. En lenguaje estadístico sería la división entre la diferencia de la característica observada menos la media de características dividido entre la raíz cuadrada de la varianza.

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Y luego la ecuación de estimación de parámetros aprendidos

$$y = \gamma x + \beta$$

Donde:

μ = media de las características

σ^2 = varianza

ϵ = pequeño valor corrector de la división por cero

γ = parámetro aprendible de escala

β = parámetro aprendible de desplazamiento

Nota: los Transformers generalmente utilizan LayerNorm que normaliza características de una representación individual y *no utiliza* BatchNorm que usa estadísticas relacionadas con el batch.

Aquí te muestro como se vería un código Python para que se entienda el tema del Add & Norm, donde x en cada se actualiza con la información de atención y luego se normaliza.

python

```
def transformer_block(x):  
    attention_output = attention(x)  
    x = x + attention_output  
    x = layer_norm(x)  
    ff_output = feed_forward(x)  
    x = x + ff_output  
    x = layer_norm(x)  
    return x
```

Si te has percatado, al leer el artículo de Vaswani (2023) artículo inicial del 2017, presenta la formulación de $LayerNorm(X + Sublayer(X))$ K , que se llama también *PostLN*. Pero, muchos en la parte de la implementación hacen lo siguiente: $x + Sublayer(LayerNorm(x))$. O sea, siguen el camino X , Atención, Addm, Norm.

Un camino diferente es un PreLN: x , Norm, Attencion, Add, x . Eso lo notarás cuando aplicar un PreLN o PostLn a un Transformers desde cero difieren en la dinámica de entrenamiento.

Para cerrar esta sección, en realidad Add y Norm aparecen dos veces en el encoder, es decir:

Primera vez

$X_1 = LayerNorm(X + MultiHeadAttention(X))$

Segunda vez

$$X_2 = \text{LayerNorm}(X_1 + \text{FFN}(X_1))$$

Por eso en el diagrama aparecen dos bloques "Add & Norm".

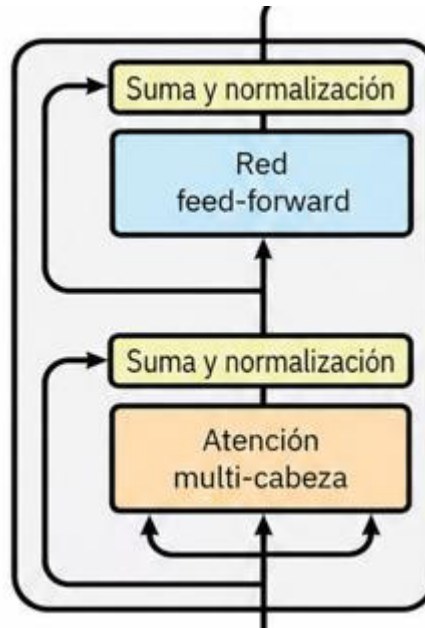


Fig. 2 ENCODER

3.12 Feed-Forward Network (FFN)

Ahora la representación entra a una red FFN, que matemáticamente se expresa así:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

La atención permite mezclar información entre diferentes posiciones y la red feed-forward realiza una transformación no lineal de la representación en cada posición.

Si has observado la Figura 1, verás también que el ENCODER, se repite N veces algo así como:

Encoder Layer 1, Encoder Layer 2, Encoder Layer 3, ..., Encoder Layer N,

donde cada capa recibe las representaciones de la capa anterior, algo así como:

Capa de relaciones básica, Capa de relaciones más complejas, Capa de relaciones semánticas, ..., Capa de representación contextual profunda.

Nota: Estas capas de relaciones son expresiones conceptuales de mi parte, no que estrictamente se cumpla que hay una Capa de Sintaxis, otra Capa de gramática u otra capa de semántica.

3.13 El objetivo del ENCODER

Apreciado lector después de tanto intento por comprender al DECODER ya llegamos hasta aquí.

Partimos con el texto:

"Se acerca el fenómeno El Niño y las casas de la costa no están preparadas para lluvias intensas"

El Encoder produce una representación matemática de toda la oración.

De hecho no produce directamente: "El Niño es un fenómeno climático..."

Pero produce algo mucho más importante para la red:

$$H = [h_1, h_2, h_3, \dots, h_{18}]$$

donde cada h_i es un vector contextualizado. Por ejemplo:

h_1 es la representación contextual de "Se"

h_2 es la representación contextual de "acerca"

h_3 es la representación contextual de "el"

h_4 es la representación contextual de "fenómeno"

h_5 es la representación contextual de "El"

h_6 es la representación contextual de "Niño"

...

h_{18} es la representación contextual de "intensas"

3.14 Codificación de un ENCODER

Especificaciones en las que se ha ejecutado el siguiente código:

Procesador Core i7 – 2.09GHz- 10g

GPU Nvidia GForce Mx-130

RAM 16Gb

python

```
# =====
# TRANSFORMER ENCODER - IMPLEMENTACIÓN EXPLICATIVA
# =====
#
# Oración de entrada:
#
# "Se acerca el fenómeno El Niño y las casas de la costa
# no están preparadas para lluvias intensas"
#
# El programa muestra:
#
# 1. Texto original
# 2. Tokenización
# 3. IDs de los tokens
# 4. Embeddings
# 5. Codificación posicional
# 6. Entrada al Encoder
# 7. Q, K y V
# 8. Scaled Dot-Product Attention
# 9. Multi-Head Attention
# 10. Add & Norm
# 11. Feed Forward Network
# 12. Segundo Add & Norm
# 13. Salida final del Encoder
#
# =====

import re
import math
import torch
import torch.nn as nn

# =====
# 0. CONFIGURACIÓN
# =====

# Fijamos la semilla para que los resultados sean reproducibles.
SEED = 42
torch.manual_seed(SEED)

print("=" * 80)
print("TRANSFORMER ENCODER - DEMOSTRACIÓN PASO A PASO")
print("=" * 80)

# Texto de entrada
texto = (
    "Se acerca el fenómeno El Niño y las casas de la costa "
    "no están preparadas para lluvias intensas"
)

print("\n[0] TEXTO DE ENTRADA")
```

```

print("-" * 80)
print(texto)

# =====
# 1. TOKENIZACIÓN
# =====

print("\n" + "=" * 80)
print("[1] TOKENIZACIÓN")
print("=" * 80)

# Para esta demostración utilizamos un tokenizer extremadamente
# sencillo: separa palabras y conserva signos de puntuación.
#
# Un tokenizer real puede dividir una palabra en subpalabras.
#
# Ejemplo:
# "preparadas" podría convertirse en:
# ["pre", "paradas"]
#
# dependiendo del algoritmo utilizado.

tokens = re.findall(r"\w+|[\^\w\s]", texto, flags=re.UNICODE)

print("Tokens encontrados:")
for i, token in enumerate(tokens):
    print(f"Posición {i:2d} -> {token}")

print(f"\nNúmero total de tokens: {len(tokens)}")

# =====
# 2. CREACIÓN DEL VOCABULARIO
# =====

print("\n" + "=" * 80)
print("[2] VOCABULARIO E IDs")
print("=" * 80)

# En un modelo real el vocabulario ya viene construido durante
# el entrenamiento.
#
# Aquí lo construimos únicamente con los tokens de nuestra oración.

vocab = {
    "<PAD>": 0,
    "<UNK>": 1
}

for token in tokens:
    if token not in vocab:
        vocab[token] = len(vocab)

print("Vocabulario:")
for palabra, indice in vocab.items():
    print(f"{palabra:15s} -> {indice}")

# Convertimos los tokens a IDs
token_ids = [vocab.get(token, vocab["<UNK>"]) for token in tokens]

print("\nSecuencia de IDs:")
print(token_ids)

```

```

# =====
# 3. PARÁMETROS DEL TRANSFORMER
# =====

print("\n" + "=" * 80)
print("[3] CONFIGURACIÓN DEL TRANSFORMER")
print("=" * 80)

# Dimensiones pequeñas para que los resultados sean fáciles
# de inspeccionar.
#
# Un Transformer real puede utilizar dimensiones mucho mayores.

d_model = 32      # dimensión de cada embedding
num_heads = 4     # número de cabezas de atención
d_ff = 64         # dimensión interna del Feed Forward
num_layers = 2    # número de capas del Encoder

seq_len = len(tokens)

print(f"Número de tokens      : {seq_len}")
print(f"Dimensión embedding    : {d_model}")
print(f"Número de cabezas       : {num_heads}")
print(f"Dimensión Feed Forward  : {d_ff}")
print(f"Número de capas Encoder: {num_layers}")

assert d_model % num_heads == 0

head_dim = d_model // num_heads

print(f"Dimensión de cada cabeza: {head_dim}")

# =====
# 4. EMBEDDING
# =====

print("\n" + "=" * 80)
print("[4] EMBEDDING")
print("=" * 80)

# Creamos la capa de embedding.
#
# Cada ID se convierte en un vector de dimensión d_model.

embedding = nn.Embedding(
    num_embeddings=len(vocab),
    embedding_dim=d_model
)

# Convertimos los IDs en tensor
input_ids = torch.tensor(token_ids, dtype=torch.long)

# Añadimos dimensión batch:
#
# [seq_len]
#   ↓
# [1, seq_len]

input_ids = input_ids.unsqueeze(0)

# Obtenemos embeddings

```

```

x = embedding(input_ids)

print(f"Forma de input_ids : {input_ids.shape}")
print(f"Forma de embeddings: {x.shape}")

print("\nLa forma es:")
print("[batch, tokens, dimensiones]")
print(x.shape)

print("\nPrimeros embeddings:")
for i in range(min(5, seq_len)):
    print(
        f"{tokens[i]:15s} -> "
        f"{x[0, i, :8].detach().numpy()}"
    )

print(
    "\nSe muestran solamente las primeras 8 dimensiones "
    "para no saturar la salida."
)

# =====
# 5. CODIFICACIÓN POSICIONAL
# =====

print("\n" + "=" * 80)
print("[5] CODIFICACIÓN POSICIONAL")
print("=" * 80)

class PositionalEncoding(nn.Module):

    def __init__(self, d_model, max_len=5000):

        super().__init__()

        # Matriz:
        #
        # [posición, dimensión]

        pe = torch.zeros(max_len, d_model)

        position = torch.arange(
            0,
            max_len,
            dtype=torch.float
        ).unsqueeze(1)

        # Frecuencias
        div_term = torch.exp(
            torch.arange(
                0,
                d_model,
                2,
                dtype=torch.float
            )
            * (-math.log(10000.0) / d_model)
        )

        # Funciones seno y coseno
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        # Añadimos dimensión batch

```

```

        pe = pe.unsqueeze(0)

        self.register_buffer("pe", pe)

    def forward(self, x):

        return x + self.pe[:, :x.size(1), :]

positional_encoding = PositionalEncoding(d_model)
x_with_position = positional_encoding(x)

print("Cada token ahora posee:")

print(
    "Embedding del token + información de su posición"
)

print("\nEjemplo:")

for i in range(min(5, seq_len)):

    print(f"\nToken: {tokens[i]}")
    print(f"Posición: {i}")

    print(
        "Embedding :",
        x[0, i, :6].detach().numpy()
    )

    print(
        "Con posición:",
        x_with_position[0, i, :6].detach().numpy()
    )

# =====
# 6. SELF-ATTENTION
# =====

print("\n" + "=" * 80)
print("[6] SELF-ATTENTION")
print("=" * 80)

class MultiHeadSelfAttention(nn.Module):

    def __init__(self, d_model, num_heads):

        super().__init__()

        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        # Proyecciones para Q, K y V

        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)

        # Proyección final después de concatenar
        # las diferentes cabezas.

```

```

self.W_O = nn.Linear(d_model, d_model)

# Guardaremos las atenciones para mostrarlas
# posteriormente.

self.attention_weights = None

def forward(self, x):

    batch_size = x.size(0)

    # -----
    # Q, K y V
    # -----

    Q = self.W_Q(x)
    K = self.W_K(x)
    V = self.W_V(x)

    print("\nFORMAS DE Q, K Y V")
    print("-----")
    print("Q:", Q.shape)
    print("K:", K.shape)
    print("V:", V.shape)

    # -----
    # Dividimos en cabezas
    # -----

    Q = Q.view(
        batch_size,
        -1,
        self.num_heads,
        self.head_dim
    ).transpose(1, 2)

    K = K.view(
        batch_size,
        -1,
        self.num_heads,
        self.head_dim
    ).transpose(1, 2)

    V = V.view(
        batch_size,
        -1,
        self.num_heads,
        self.head_dim
    ).transpose(1, 2)

    print("\nDESPUÉS DE DIVIDIR EN CABEZAS")
    print("-----")

    print("Q:", Q.shape)
    print("K:", K.shape)
    print("V:", V.shape)

    # Forma:
    #
    # [batch, heads, tokens, head_dim]

    # -----
    # QK^T
    # -----

```

```

scores = torch.matmul(
    Q,
    K.transpose(-2, -1)
)

print("\nMATRIZ QK^T")
print("-----")
print("Forma:", scores.shape)

# -----
# Escalamiento
# -----

scores = scores / math.sqrt(self.head_dim)

# -----
# Softmax
# -----

attention = torch.softmax(
    scores,
    dim=-1
)

self.attention_weights = attention.detach()

print("\nMATRIZ DE ATENCIÓN DESPUÉS DE SOFTMAX")
print("-----")

print(
    "Forma:",
    attention.shape
)

# -----
# Attention × V
# -----

context = torch.matmul(
    attention,
    V
)

print("\nCONTEXT")
print("-----")
print("Forma:", context.shape)

# -----
# Concatenamos las cabezas
# -----

context = context.transpose(
    1,
    2
).contiguous()

context = context.view(
    batch_size,
    -1,
    self.d_model
)

# -----

```

```

# Proyección final
# -----

output = self.W_O(context)

print("\nSALIDA MULTI-HEAD ATTENTION")
print("-----")
print("Forma:", output.shape)

return output

# Creamos la atención
attention_layer = MultiHeadSelfAttention(
    d_model,
    num_heads
)

# Ejecutamos Self-Attention
attention_output = attention_layer(
    x_with_position
)

# =====
# 7. INSPECCIONAR LOS PESOS DE ATENCIÓN
# =====

print("\n" + "=" * 80)
print("[7] ¿A QUÉ TOKENS PRESTA ATENCIÓN EL MODELO?")
print("=" * 80)

attention_weights = attention_layer.attention_weights

# Tomamos la primera muestra del batch
# y la primera cabeza.

head = 0

attention_matrix = attention_weights[0, head]

print(
    f"\nMostrando la matriz de atención de la cabeza {head + 1}"
)

print(
    "\nFilas = token que está consultando"
    "\nColumnas = tokens que está observando"
)

# Imprimimos una tabla simplificada

print("\n")

for i, token in enumerate(tokens):

    pesos = attention_matrix[i]

    # Obtener los 3 tokens con mayor atención
    valores, indices = torch.topk(
        pesos,
        k=min(3, len(tokens))
    )

```



```

print(f"{token:15s} presta atención a:")

for valor, indice in zip(valores, indices):

    print(
        f"    -> {tokens[indice]:15s} "
        f"{valor.item():.4f}"
    )

# =====
# 8. ADD & NORM
# =====

print("\n" + "=" * 80)
print("[8] ADD & NORM DESPUÉS DE ATTENTION")
print("=" * 80)

# Implementamos exactamente la idea del Transformer clásico:
#
# output = LayerNorm(x + Attention(x))

layer_norm_1 = nn.LayerNorm(d_model)

residual_1 = x_with_position + attention_output

normalized_1 = layer_norm_1(
    residual_1
)

print("Entrada original:")
print(
    "Forma:",
    x_with_position.shape
)

print("\nSalida de Attention:")
print(
    "Forma:",
    attention_output.shape
)

print("\nDespués de ADD:")
print(
    "X + Attention(X) "
)

print(
    "Forma:",
    residual_1.shape
)

print("\nDespués de LayerNorm:")
print(
    "Forma:",
    normalized_1.shape
)

# =====
# 9. FEED FORWARD NETWORK
# =====

```

```

print("\n" + "=" * 80)
print("[9] FEED FORWARD NETWORK")
print("=" * 80)

class FeedForward(nn.Module):

    def __init__(self, d_model, d_ff):

        super().__init__()

        self.linear1 = nn.Linear(
            d_model,
            d_ff
        )

        self.activation = nn.GELU()

        self.linear2 = nn.Linear(
            d_ff,
            d_model
        )

    def forward(self, x):

        x = self.linear1(x)

        print("\nDespués de Linear 1:")
        print(x.shape)

        x = self.activation(x)

        print("\nDespués de GELU:")
        print(x.shape)

        x = self.linear2(x)

        print("\nDespués de Linear 2:")
        print(x.shape)

        return x

feed_forward = FeedForward(
    d_model,
    d_ff
)

ff_output = feed_forward(
    normalized_1
)

# =====
# 10. SEGUNDO ADD & NORM
# =====

print("\n" + "=" * 80)
print("[10] SEGUNDO ADD & NORM")
print("=" * 80)

layer_norm_2 = nn.LayerNorm(d_model)

residual_2 = (
    normalized_1 +

```

```

        ff_output
    )

    encoder_layer_output = layer_norm_2(
        residual_2
    )

    print("Entrada al Feed Forward:")
    print(
        normalized_1.shape
    )

    print("\nSalida del Feed Forward:")
    print(
        ff_output.shape
    )

    print("\nDespués de ADD:")
    print(
        "X + FeedForward(X) "
    )

    print(
        residual_2.shape
    )

    print("\nDespués de LayerNorm:")

    print(
        encoder_layer_output.shape
    )

# =====
# 11. ENCODER COMPLETO
# =====

print("\n" + "=" * 80)
print("[11] ENCODER COMPLETO")
print("=" * 80)

class EncoderLayer(nn.Module):

    def __init__(
        self,
        d_model,
        num_heads,
        d_ff
    ):

        super().__init__()

        self.attention = MultiHeadSelfAttention(
            d_model,
            num_heads
        )

        self.norm1 = nn.LayerNorm(
            d_model
        )

        self.feed_forward = FeedForward(
            d_model,

```

```

        d_ff
    )

    self.norm2 = nn.LayerNorm(
        d_model
    )

def forward(self, x, layer_number):

    print("\n")
    print("#" * 80)
    print(
        f"ENCODER LAYER {layer_number}"
    )
    print("#" * 80)

    # -----
    # 1. Self-Attention
    # -----

    attention_output = self.attention(x)

    # -----
    # 2. Add & Norm
    # -----

    x = self.norm1(
        x + attention_output
    )

    print(
        "\n[Add & Norm 1] ->",
        x.shape
    )

    # -----
    # 3. Feed Forward
    # -----

    ff_output = self.feed_forward(x)

    # -----
    # 4. Add & Norm
    # -----

    x = self.norm2(
        x + ff_output
    )

    print(
        "\n[Add & Norm 2] ->",
        x.shape
    )

    return x

# Creamos las N capas
encoder_layers = nn.ModuleList(
    [
        EncoderLayer(
            d_model,
            num_heads,
            d_ff

```

```

    )
    for _ in range(num_layers)
    ]
)

# Entrada al Encoder
encoder_output = x_with_position

# Pasamos por todas las capas
for layer_number, layer in enumerate(
    encoder_layers,
    start=1
):

    encoder_output = layer(
        encoder_output,
        layer_number
    )

# =====
# 12. SALIDA FINAL DEL ENCODER
# =====

print("\n" + "=" * 80)
print("[12] SALIDA FINAL DEL ENCODER")
print("=" * 80)

print(
    "\nForma final:",
    encoder_output.shape
)

print(
    "\nInterpretación:"
)

print(
    "Cada token ahora está representado mediante un vector "
    "contextualizado de dimensión",
    d_model
)

print(
    "\nPor ejemplo:"
)

for i, token in enumerate(tokens):

    vector = encoder_output[
        0,
        i
    ]

    print(
        f"{i:2d} | "
        f"{token:15s} | "
        f"{vector[:6].detach().numpy()}"
    )

```

```

# =====
# 13. RESUMEN DEL FLUJO
# =====

print("\n" + "=" * 80)
print("[13] RESUMEN DEL PROCESAMIENTO")
print("=" * 80)

print(
    """
    Texto
    ↓
    Tokenización
    ↓
    Tokens
    ↓
    IDs
    ↓
    Embedding
    ↓
    Codificación Posicional
    ↓
    -----
    ENCODER LAYER 1
    ↓
    Multi-Head Self-Attention
    ↓
    Add & Norm
    ↓
    Feed Forward
    ↓
    Add & Norm
    -----
    ↓
    ENCODER LAYER 2
    ↓
    Multi-Head Self-Attention
    ↓
    Add & Norm
    ↓
    Feed Forward
    ↓
    Add & Norm
    -----
    ↓
    SALIDA DEL ENCODER
    """
)

print(
    "\nEl Encoder ha transformado la oración original "
    "en una matriz de representaciones contextuales."
)

print(
    "\nDimensión final:",
    encoder_output.shape
)

print(
    "\nFin de la demostración."
)

```

Salida:

```
Salida pyhton
=====
TRANSFORMER ENCODER - DEMOSTRACIÓN PASO A PASO
=====

[0] TEXTO DE ENTRADA
=====
Se acerca el fenómeno El Niño y las casas de la costa no están preparadas para lluvias intensas

=====

[1] TOKENIZACIÓN
=====
Tokens encontrados:
Posición 0 -> Se
Posición 1 -> acerca
Posición 2 -> el
Posición 3 -> fenómeno
Posición 4 -> El
Posición 5 -> Niño
Posición 6 -> y
Posición 7 -> las
Posición 8 -> casas
Posición 9 -> de
Posición 10 -> la
Posición 11 -> costa
Posición 12 -> no
Posición 13 -> están
Posición 14 -> preparadas
Posición 15 -> para
Posición 16 -> lluvias
Posición 17 -> intensas

Número total de tokens: 18

=====

[2] VOCABULARIO E IDs
=====
Vocabulario:
<PAD> -> 0
<UNK> -> 1
Se -> 2
acerca -> 3
el -> 4
fenómeno -> 5
El -> 6
Niño -> 7
y -> 8
las -> 9
casas -> 10
de -> 11
la -> 12
costa -> 13
no -> 14
están -> 15
preparadas -> 16
para -> 17
lluvias -> 18
intensas -> 19

Secuencia de IDs:
[2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

=====

[3] CONFIGURACIÓN DEL TRANSFORMER
=====
Número de tokens : 18
Dimensión embedding : 32
Número de cabezas : 4
Dimensión Feed Forward : 64
Número de capas Encoder: 2
Dimensión de cada cabeza: 8

=====

[4] EMBEDDING
```

```

=====
Forma de input_ids : torch.Size([1, 18])
Forma de embeddings: torch.Size([1, 18, 32])

La forma es:
[batch, tokens, dimensiones]
torch.Size([1, 18, 32])

Primeros embeddings:
Se      -> [ 1.4451338  0.85641253  2.2180758  0.5231655  0.34664667 -0.19733144
-1.0545889  1.2779956 ]
acerca  -> [-2.5095444  0.48800105  0.78458685  0.02864719  0.6407553  0.5832474
1.0669267 -0.45015338]
el      -> [ 1.9311606  1.0118638 -1.4364065 -1.1298598 -0.13603453 1.6354096
0.65474075 0.57600456]
fenómeno -> [-1.9005532  0.22857653  0.0248594 -0.34595025 0.2868328 -0.7308424
0.17482026 -1.0939293 ]
El      -> [ 1.7743884 -0.921551  0.9624499 -0.33701515 -1.1753336 0.35805708
0.4787679 1.3537 ]

```

Se muestran solamente las primeras 8 dimensiones para no saturar la salida.

[5] CODIFICACIÓN POSICIONAL

Cada token ahora posee:
 Embedding **del** token + información de su posición

Ejemplo:

```

Token: Se
Posición: 0
Embedding : [ 1.4451338  0.85641253  2.2180758  0.5231655  0.34664667 -0.19733144]
Con posición: [1.4451338 1.8564125 2.2180758 1.5231655 0.34664667 0.8026686 ]

Token: acerca
Posición: 1
Embedding : [-2.5095444  0.48800105  0.78458685  0.02864719  0.6407553  0.5832474 ]
Con posición: [-1.6680734  1.0283034  1.3177552  0.87465626  0.9517389  1.5336628 ]

Token: el
Posición: 2
Embedding : [ 1.9311606  1.0118638 -1.4364065 -1.1298598 -0.13603453 1.6354096 ]
Con posición: [ 2.840458  0.59571695 -0.53427577 -0.698397 0.45509255 2.441988 ]

Token: fenómeno
Posición: 3
Embedding : [-1.9005532  0.22857653  0.0248594 -0.34595025 0.2868328 -0.7308424 ]
Con posición: [-1.7594333 -0.76141596 1.0181125 -0.46191642 1.0994817 -0.14808875]

Token: El
Posición: 4
Embedding : [ 1.7743884 -0.921551  0.9624499 -0.33701515 -1.1753336 0.35805708]
Con posición: [ 1.017586 -1.5751946 1.7409216 -0.96469486 -0.22175288 0.6591946 ]

```

[6] SELF-ATTENTION

FORMAS DE Q, K Y V

```

-----
Q: torch.Size([1, 18, 32])
K: torch.Size([1, 18, 32])
V: torch.Size([1, 18, 32])

```

DESPUÉS DE DIVIDIR EN CABEZAS

```

-----
Q: torch.Size([1, 4, 18, 8])
K: torch.Size([1, 4, 18, 8])
V: torch.Size([1, 4, 18, 8])

```

MATRIZ QK[^]T

```

-----
Forma: torch.Size([1, 4, 18, 18])

```

MATRIZ DE ATENCIÓN DESPUÉS DE SOFTMAX

```

-----
Forma: torch.Size([1, 4, 18, 18])

```

CONTEXT

Forma: torch.Size([1, 4, 18, 8])

SALIDA MULTI-HEAD ATTENTION

Forma: torch.Size([1, 18, 32])

=====

[7] ¿A QUÉ TOKENS PRESTA ATENCIÓN EL MODELO?

=====

Mostrando la matriz de atención de la cabeza 1

Filas = token que está consultando
Columnas = tokens que está observando

Se	presta atención a:
-> casas	0.1512
-> el	0.1133
-> para	0.0881
acerca	presta atención a:
-> casas	0.0853
-> acerca	0.0681
-> el	0.0680
el	presta atención a:
-> para	0.1110
-> el	0.1077
-> la	0.1036
fenómeno	presta atención a:
-> y	0.0829
-> el	0.0790
-> Niño	0.0782
El	presta atención a:
-> casas	0.1342
-> el	0.1208
-> de	0.0896
Niño	presta atención a:
-> casas	0.0753
-> costa	0.0730
-> preparadas	0.0694
y	presta atención a:
-> el	0.1265
-> casas	0.1143
-> preparadas	0.0743
las	presta atención a:
-> intensas	0.1073
-> la	0.0798
-> fenómeno	0.0763
casas	presta atención a:
-> el	0.0991
-> casas	0.0983
-> Se	0.0891
de	presta atención a:
-> casas	0.0861
-> para	0.0850
-> la	0.0796
la	presta atención a:
-> costa	0.0708
-> para	0.0670
-> fenómeno	0.0659
costa	presta atención a:
-> intensas	0.0987
-> de	0.0831
-> fenómeno	0.0803
no	presta atención a:
-> fenómeno	0.1215
-> casas	0.0917
-> de	0.0863
están	presta atención a:
-> costa	0.0825
-> para	0.0791
-> lluvias	0.0791
preparadas	presta atención a:
-> costa	0.0685
-> Se	0.0677
-> no	0.0667
para	presta atención a:
-> fenómeno	0.1048
-> de	0.0970

```

-> no          0.0897
lluvias        presta atención a:
-> las         0.1552
-> de          0.1004
-> intensas    0.0827
intensas       presta atención a:
-> casas       0.0845
-> fenómeno    0.0832
-> El          0.0743

```

[8] ADD & NORM DESPUÉS DE ATTENTION

```

Entrada original:
Forma: torch.Size([1, 18, 32])

Salida de Attention:
Forma: torch.Size([1, 18, 32])

Después de ADD:
X + Attention(X)
Forma: torch.Size([1, 18, 32])

Después de LayerNorm:
Forma: torch.Size([1, 18, 32])

```

[9] FEED FORWARD NETWORK

```

Después de Linear 1:
torch.Size([1, 18, 64])

Después de GELU:
torch.Size([1, 18, 64])

Después de Linear 2:
torch.Size([1, 18, 32])

```

[10] SEGUNDO ADD & NORM

```

Entrada al Feed Forward:
torch.Size([1, 18, 32])

Salida del Feed Forward:
torch.Size([1, 18, 32])

Después de ADD:
X + FeedForward(X)
torch.Size([1, 18, 32])

Después de LayerNorm:
torch.Size([1, 18, 32])

```

[11] ENCODER COMPLETO

```

#####
ENCODER LAYER 1
#####

FORMAS DE Q, K Y V
-----
Q: torch.Size([1, 18, 32])
K: torch.Size([1, 18, 32])
V: torch.Size([1, 18, 32])

DESPUÉS DE DIVIDIR EN CABEZAS
-----
Q: torch.Size([1, 4, 18, 8])
K: torch.Size([1, 4, 18, 8])
V: torch.Size([1, 4, 18, 8])

MATRIZ QK^T
-----
Forma: torch.Size([1, 4, 18, 18])

```

```

MATRIZ DE ATENCIÓN DESPUÉS DE SOFTMAX
-----
Forma: torch.Size([1, 4, 18, 18])

CONTEXT
-----
Forma: torch.Size([1, 4, 18, 8])

SALIDA MULTI-HEAD ATTENTION
-----
Forma: torch.Size([1, 18, 32])

[Add & Norm 1] -> torch.Size([1, 18, 32])

Después de Linear 1:
torch.Size([1, 18, 64])

Después de GELU:
torch.Size([1, 18, 64])

Después de Linear 2:
torch.Size([1, 18, 32])

[Add & Norm 2] -> torch.Size([1, 18, 32])

#####
ENCODER LAYER 2
#####

FORMAS DE Q, K Y V
-----
Q: torch.Size([1, 18, 32])
K: torch.Size([1, 18, 32])
V: torch.Size([1, 18, 32])

DESPUÉS DE DIVIDIR EN CABEZAS
-----
Q: torch.Size([1, 4, 18, 8])
K: torch.Size([1, 4, 18, 8])
V: torch.Size([1, 4, 18, 8])

MATRIZ QK^T
-----
Forma: torch.Size([1, 4, 18, 18])

MATRIZ DE ATENCIÓN DESPUÉS DE SOFTMAX
-----
Forma: torch.Size([1, 4, 18, 18])

CONTEXT
-----
Forma: torch.Size([1, 4, 18, 8])

SALIDA MULTI-HEAD ATTENTION
-----
Forma: torch.Size([1, 18, 32])

[Add & Norm 1] -> torch.Size([1, 18, 32])

Después de Linear 1:
torch.Size([1, 18, 64])

Después de GELU:
torch.Size([1, 18, 64])

Después de Linear 2:
torch.Size([1, 18, 32])

[Add & Norm 2] -> torch.Size([1, 18, 32])

=====
[12] SALIDA FINAL DEL ENCODER
=====

Forma final: torch.Size([1, 18, 32])

Interpretación:
Cada token ahora está representado mediante un vector contextualizado de dimensión 32

```

```

Por ejemplo:
0 | Se                | [-0.12034762  0.94876134  2.2267635  0.6754224 -0.8459216  0.23835148]
1 | acerca            | [-2.3569376  -0.49594778  1.6431907  -0.08413047  0.26047185  0.91686714]
2 | el                | [ 0.67566997 -0.71666676  0.07616617 -1.1069497  -0.5261292  1.3393713 ]
3 | fenómeno          | [-1.624873  -1.4518403  1.2501254  -0.38401693  0.42520106  0.19040368]
4 | El                | [-0.04059545 -2.058713  1.6257999  -0.8379057  -0.70370734  0.23095319]
5 | Niño              | [-1.2594285  0.4423971  1.1544814  -2.0485384  -0.21980965 -0.3823545 ]
6 | y                 | [-1.2428963  -0.09166587 -0.5906  -2.2425559  -0.93015367 -0.28498346]
7 | las               | [ 0.4271987  -1.0338058  -0.29503956 -0.7541047  0.22822477 -0.9440847 ]
8 | casas             | [ 0.36001638 -2.049433  0.79638565 -2.143324  0.21790186 -0.9631992 ]
9 | de                | [-1.3382914  -1.1964356  0.18028517 -0.50087416 -0.6394195  -1.1166556 ]
10 | la                | [-0.44589  -1.7742131  0.155157  0.7289209  0.9832759 -2.0586717]
11 | costa             | [-1.08715  -2.0252554  0.12979886  1.1373384  -1.1303512  -1.6181253 ]
12 | no                | [-1.8647783  0.1072702  0.38844106  0.62055784 -0.88108087 -1.9223362 ]
13 | están             | [ 0.9502312  0.8834281  1.4901023  -0.44955793 -1.6916634  -0.96054566]
14 | preparadas        | [-1.058859  -0.44630873  1.427207  -0.27429014  0.15546662 -0.42176488]
15 | para              | [-1.5210897  -1.6388398  -0.02187636  0.19747332 -1.5153171  -0.8050042 ]
16 | lluvias           | [-1.2717241  -2.2679274  1.5242492  -1.2241601  -2.1419353  -0.08108476]
17 | intensas          | [-2.4377193  0.42761245 -0.18095468 -0.4308372  -1.3954062  -1.4997356 ]

```

[13] RESUMEN DEL PROCESAMIENTO

```

Texto
↓
Tokenización
↓
Tokens
↓
IDs
↓
Embedding
↓
Codificación Posicional
↓

```

```

ENCODER LAYER 1
↓
Multi-Head Self-Attention
↓
Add & Norm
↓
Feed Forward
↓
Add & Norm

```

```

ENCODER LAYER 2
↓
Multi-Head Self-Attention
↓
Add & Norm
↓
Feed Forward
↓
Add & Norm

```

SALIDA DEL ENCODER

El Encoder ha transformado la oración original en una matriz de representaciones contextuales.

Dimensión final: `torch.Size([1, 18, 32])`

Fin de la demostración.

PS G:\2025-Libros\InteligenciaArtificial\pythonCode> & C:\anaconda3\python.exe g:/2025-Libros/InteligenciaArtificial/pythonCode/Encoder.py

TRANSFORMER ENCODER - DEMOSTRACIÓN PASO A PASO

[0] TEXTO DE ENTRADA

Se acerca el fenómeno El Niño y las casas de la costa no están preparadas para lluvias intensas

[1] TOKENIZACIÓN

```
Tokens encontrados:
Posición 0 -> Se
Posición 1 -> acerca
Posición 2 -> el
Posición 3 -> fenómeno
Posición 4 -> El
Posición 5 -> Niño
Posición 6 -> y
Posición 7 -> las
Posición 8 -> casas
Posición 9 -> de
Posición 10 -> la
Posición 11 -> costa
Posición 12 -> no
Posición 13 -> están
Posición 14 -> preparadas
Posición 15 -> para
Posición 16 -> lluvias
Posición 17 -> intensas

Número total de tokens: 18

=====
[2] VOCABULARIO E IDs
=====
Vocabulario:
<PAD> -> 0
<UNK> -> 1
Se -> 2
acerca -> 3
el -> 4
fenómeno -> 5
El -> 6
Niño -> 7
y -> 8
las -> 9
casas -> 10
de -> 11
la -> 12
costa -> 13
no -> 14
están -> 15
preparadas -> 16
para -> 17
lluvias -> 18
intensas -> 19

Secuencia de IDs:
[2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

=====
[3] CONFIGURACIÓN DEL TRANSFORMER
=====
Número de tokens : 18
Dimensión embedding : 32
Número de cabezas : 4
Dimensión Feed Forward : 64
Número de capas Encoder: 2
Dimensión de cada cabeza: 8

=====
[4] EMBEDDING
=====
Forma de input_ids : torch.Size([1, 18])
Forma de embeddings: torch.Size([1, 18, 32])

La forma es:
[batch, tokens, dimensiones]
torch.Size([1, 18, 32])

Primeros embeddings:
Se -> [ 1.4451338 0.85641253 2.2180758 0.5231655 0.34664667 -0.19733144
-1.0545889 1.2779956 ]
acerca -> [-2.5095444 0.48800105 0.78458685 0.02864719 0.6407553 0.5832474
1.0669267 -0.45015338]
el -> [ 1.9311606 1.0118638 -1.4364065 -1.1298598 -0.13603453 1.6354096
0.65474075 0.57600456]
fenómeno -> [-1.9005532 0.22857653 0.0248594 -0.34595025 0.2868328 -0.7308424
0.17482026 -1.0939293 ]
El -> [ 1.7743884 -0.921551 0.9624499 -0.33701515 -1.1753336 0.35805708
```

```
0.4787679 1.3537 ]
```

Se muestran solamente las primeras 8 dimensiones para no saturar la salida.

[5] CODIFICACIÓN POSICIONAL

Cada token ahora posee:
Embedding **del** token + información de su posición

Ejemplo:

Token: Se

Posición: 0

Embedding : [1.4451338 0.85641253 2.2180758 0.5231655 0.34664667 -0.19733144]

Con posición: [1.4451338 1.8564125 2.2180758 1.5231655 0.34664667 0.8026686]

Token: acerca

Posición: 1

Embedding : [-2.5095444 0.48800105 0.78458685 0.02864719 0.6407553 0.5832474]

Con posición: [-1.6680734 1.0283034 1.3177552 0.87465626 0.9517389 1.5336628]

Token: el

Posición: 2

Embedding : [1.9311606 1.0118638 -1.4364065 -1.1298598 -0.13603453 1.6354096]

Con posición: [2.840458 0.59571695 -0.53427577 -0.698397 0.45509255 2.441988]

Token: fenómeno

Posición: 3

Embedding : [-1.9005532 0.22857653 0.0248594 -0.34595025 0.2868328 -0.7308424]

Con posición: [-1.7594333 -0.76141596 1.0181125 -0.46191642 1.0994817 -0.14808875]

Token: El

Posición: 4

Embedding : [1.7743884 -0.921551 0.9624499 -0.33701515 -1.1753336 0.35805708]

Con posición: [1.017586 -1.5751946 1.7409216 -0.96469486 -0.22175288 0.6591946]

[6] SELF-ATTENTION

FORMAS DE Q, K Y V

Q: torch.Size([1, 18, 32])

K: torch.Size([1, 18, 32])

V: torch.Size([1, 18, 32])

DESPUÉS DE DIVIDIR EN CABEZAS

Q: torch.Size([1, 4, 18, 8])

K: torch.Size([1, 4, 18, 8])

V: torch.Size([1, 4, 18, 8])

MATRIZ QK^T

Forma: torch.Size([1, 4, 18, 18])

MATRIZ DE ATENCIÓN DESPUÉS DE SOFTMAX

Forma: torch.Size([1, 4, 18, 18])

CONTEXT

Forma: torch.Size([1, 4, 18, 8])

SALIDA MULTI-HEAD ATTENTION

Forma: torch.Size([1, 18, 32])

[7] ¿A QUÉ TOKENS PRESTA ATENCIÓN EL MODELO?

Mostrando la matriz de atención de la cabeza 1

Filas = token que está consultando

Columnas = tokens que está observando

Se presta atención a:

```
-> casas      0.1512
-> el         0.1133
-> para       0.0881
acerca        presta atención a:
-> casas      0.0853
-> acerca     0.0681
-> el         0.0680
el            presta atención a:
-> para       0.1110
-> el         0.1077
-> la         0.1036
fenómeno      presta atención a:
-> y          0.0829
-> el         0.0790
-> Niño       0.0782
El            presta atención a:
-> casas      0.1342
-> el         0.1208
-> de         0.0896
Niño          presta atención a:
-> casas      0.0753
-> costa      0.0730
-> preparadas 0.0694
y             presta atención a:
-> el         0.1265
-> casas      0.1143
-> preparadas 0.0743
las           presta atención a:
-> intensas   0.1073
-> la         0.0798
-> fenómeno   0.0763
casas         presta atención a:
-> el         0.0991
-> casas      0.0983
-> Se         0.0891
de            presta atención a:
-> casas      0.0861
-> para       0.0850
-> la         0.0796
la            presta atención a:
-> costa      0.0708
-> para       0.0670
-> fenómeno   0.0659
costa         presta atención a:
-> intensas   0.0987
-> de         0.0831
-> fenómeno   0.0803
no            presta atención a:
-> fenómeno   0.1215
-> casas      0.0917
-> de         0.0863
están         presta atención a:
-> costa      0.0825
-> para       0.0791
-> lluvias    0.0791
preparadas    presta atención a:
-> costa      0.0685
-> Se         0.0677
-> no         0.0667
para          presta atención a:
-> fenómeno   0.1048
-> de         0.0970
-> no         0.0897
lluvias       presta atención a:
-> las        0.1552
-> de         0.1004
-> intensas   0.0827
intensas      presta atención a:
-> casas      0.0845
-> fenómeno   0.0832
-> El         0.0743

=====
[8] ADD & NORM DESPUÉS DE ATTENTION
=====

Entrada original:
Forma: torch.Size([1, 18, 32])

Salida de Attention:
Forma: torch.Size([1, 18, 32])
```

Después de ADD:
X + Attention(X)
Forma: torch.Size([1, 18, 32])

Después de LayerNorm:
Forma: torch.Size([1, 18, 32])

[9] FEED FORWARD NETWORK

Después de Linear 1:
torch.Size([1, 18, 64])

Después de GELU:
torch.Size([1, 18, 64])

Después de Linear 2:
torch.Size([1, 18, 32])

[10] SEGUNDO ADD & NORM

Entrada al Feed Forward:
torch.Size([1, 18, 32])

Salida del Feed Forward:
torch.Size([1, 18, 32])

Después de ADD:
X + FeedForward(X)
torch.Size([1, 18, 32])

Después de LayerNorm:
torch.Size([1, 18, 32])

[11] ENCODER COMPLETO

ENCODER LAYER 1
#####

FORMAS DE Q, K Y V

Q: torch.Size([1, 18, 32])
K: torch.Size([1, 18, 32])
V: torch.Size([1, 18, 32])

DESPUÉS DE DIVIDIR EN CABEZAS

Q: torch.Size([1, 4, 18, 8])
K: torch.Size([1, 4, 18, 8])
V: torch.Size([1, 4, 18, 8])

MATRIZ QK^T

Forma: torch.Size([1, 4, 18, 18])

MATRIZ DE ATENCIÓN DESPUÉS DE SOFTMAX

Forma: torch.Size([1, 4, 18, 18])

CONTEXT

Forma: torch.Size([1, 4, 18, 8])

SALIDA MULTI-HEAD ATTENTION

Forma: torch.Size([1, 18, 32])

[Add & Norm 1] -> torch.Size([1, 18, 32])

Después de Linear 1:
torch.Size([1, 18, 64])

Después de GELU:


```

torch.Size([1, 18, 64])

Después de Linear 2:
torch.Size([1, 18, 32])

[Add & Norm 2] -> torch.Size([1, 18, 32])

#####
ENCODER LAYER 2
#####

FORMAS DE Q, K Y V
-----
Q: torch.Size([1, 18, 32])
K: torch.Size([1, 18, 32])
V: torch.Size([1, 18, 32])

DESPUÉS DE DIVIDIR EN CABEZAS
-----
Q: torch.Size([1, 4, 18, 8])
K: torch.Size([1, 4, 18, 8])
V: torch.Size([1, 4, 18, 8])

MATRIZ QK^T
-----
Forma: torch.Size([1, 4, 18, 18])

MATRIZ DE ATENCIÓN DESPUÉS DE SOFTMAX
-----
Forma: torch.Size([1, 4, 18, 18])

CONTEXT
-----
Forma: torch.Size([1, 4, 18, 8])

SALIDA MULTI-HEAD ATTENTION
-----
Forma: torch.Size([1, 18, 32])

[Add & Norm 1] -> torch.Size([1, 18, 32])

Después de Linear 1:
torch.Size([1, 18, 64])

Después de GELU:
torch.Size([1, 18, 64])

Después de Linear 2:
torch.Size([1, 18, 32])

[Add & Norm 2] -> torch.Size([1, 18, 32])

=====
[12] SALIDA FINAL DEL ENCODER
=====

Forma final: torch.Size([1, 18, 32])

Interpretación:
Cada token ahora está representado mediante un vector contextualizado de dimensión 32

Por ejemplo:
0 | Se                | [-0.12034762  0.94876134  2.2267635  0.6754224 -0.8459216  0.23835148]
1 | acerca            | [-2.3569376 -0.49594778  1.6431907 -0.08413047  0.26047185  0.91686714]
2 | el                | [ 0.67566997 -0.71666676  0.07616617 -1.1069497 -0.5261292  1.3393713 ]
3 | fenómeno          | [-1.624873 -1.4518403  1.2501254 -0.38401693  0.42520106  0.19040368]
4 | El               | [-0.04059545 -2.058713  1.6257999 -0.8379057 -0.70370734  0.23095319]
5 | Niño             | [-1.2594285  0.4423971  1.1544814 -2.0485384 -0.21980965 -0.3823545 ]
6 | y                | [-1.2428963 -0.09166587 -0.5906 -2.2425559 -0.93015367 -0.28498346]
7 | las              | [ 0.4271987 -1.0338058 -0.29503956 -0.7541047  0.22822477 -0.9440847 ]
8 | casas            | [ 0.36001638 -2.049433  0.79638565 -2.143324  0.21790186 -0.9631992 ]
9 | de               | [-1.3382914 -1.1964356  0.18028517 -0.50087416 -0.6394195 -1.1166556 ]
10 | la               | [-0.44589 -1.7742131  0.155157  0.7289209  0.9832759 -2.0586717]
11 | costa            | [-1.08715 -2.0252554  0.12979886  1.1373384 -1.1303512 -1.6181253 ]
12 | no               | [-1.8647783  0.1072702  0.38844106  0.62055784 -0.88108087 -1.9223362 ]
13 | están            | [ 0.9502312  0.8834281  1.4901023 -0.44955793 -1.6916634 -0.96054566]
14 | preparadas       | [-1.058859 -0.44630873  1.427207 -0.27429014  0.15546662 -0.42176488]
15 | para             | [-1.5210897 -1.6388398 -0.02187636  0.19747332 -1.5153171 -0.8050042 ]
16 | lluvias          | [-1.2717241 -2.2679274  1.5242492 -1.2241601 -2.1419353 -0.08108476]

```

```
17 | intensas | [-2.4377193 0.42761245 -0.18095468 -0.4308372 -1.3954062 -1.4997356 ]

=====
[13] RESUMEN DEL PROCESAMIENTO
=====

Texto
↓
Tokenización
↓
Tokens
↓
IDs
↓
Embedding
↓
Codificación Posicional
↓
-----
ENCODER LAYER 1
↓
Multi-Head Self-Attention
↓
Add & Norm
↓
Feed Forward
↓
Add & Norm
-----
↓
ENCODER LAYER 2
↓
Multi-Head Self-Attention
↓
Add & Norm
↓
Feed Forward
↓
Add & Norm
-----
↓
SALIDA DEL ENCODER

El Encoder ha transformado la oración original en una matriz de representaciones contextuales.
Dimensión final: torch.Size([1, 18, 32])
Fin de la demostración.
```

3.15 EL DECODER

Una vez más, si observas la Fig. 1 el Transformer muestra un modelo encoder - decoder. Ahora, vamos a usar nuestra oración como entrada para producir una traducción. Nuestro deseo es traducir al inglés el siguiente texto:

"Se acerca el fenómeno El Niño y las casas de la costa no están preparadas para lluvias intensas"

“The El Niño phenomenon is approaching, and the coastal houses are not prepared for intense rainfall.”

El decoder no recibe toda la respuesta “en una”, sino que la genera progresivamente. Primero parte con un Token de partida por ejemplo el token <START> añadido a ello el Decoder tiene acceso a la información producida por el Encoder. Hasta el momento su objetivo es calcular ¿El token inicial?, ¿Cómo llegaría al token “The”?.

3.16 Masked Self-Attention

Para comprender esta sección recordemos apreciados lectores que en Self-Attention, hay una secuencia de Tokens, donde cada Token puede prestar atención a cualquier otro token de la secuencia. Por ejemplo:

“Se acerca el fenómeno El Niño”

Tabla1.

Tokens que se puede consultar

	Token2	Token3	Token4	Token5	Token6
Se	si	si	si	si	si
acerca	si	si	si	si	si
el	si	si	si	si	si
fenómeno	si	si	si	si	si
El	si	si	si	si	si
Niño	si	si	si	si	si

Ya, esto funciona bien para la definición de Encoder, porque ya sabemos que el Encoder “de una” recibe toda la oración simultáneamente.

Pero el Decoder no funciona como el Encoder. Por ejemplo si deseamos que Transformer genere:

“The El Niño phenomenon is approaching...”

El modelo debe generar la oración de izquierda a derecha. La oración inicial sería:

The El Niño ¿?

Tajantemente, está prohibido que consulte las palabras que vienen después, porque todavía no las ha generado, Y si pudiera verlos ya tendría la respuesta, pero eso no es el concepto o la idea.

Entonces es aquí donde aparece la máscara sobre la matriz de atención. Vamos a ver un ejemplo: Tenemos 4 tokens

Tabla2.

Tokens que se puede consultar

	The	El	Niño	phenomenon
The	si	NO	NO	NO
El	si	si	NO	NO
Niño	si	si	si	NO
phenomenon	si	si	si	si

Te da cuenta, cada token puede mirarse y mirar hacia atrás, pero jamás puede mirar hacia adelante, concepto diferente al Encoder.

De la fórmula ya conocida:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Se calcula

$$Scores = \frac{QK^T}{\sqrt{d_k}}$$

Tabla3.

Tabla de atención

	The	El	Niño	phenomenon
The	2.1	1.3	0.8	2.4
El	1.2	3.5	1.1	2.8
Niño	0.9	1.7	4.1	3.2
phenomenon	1.4	2.1	2.7	4.5

Hasta allí son los caculos de atención que ya estamos familiarizados, ahora se aplica la máscara donde a las posiciones futuras se le asigna $-\infty$ (un valor infinitamente negativo). De manera que al aplicar softmax $e^{-\infty}$ tiende a cero, donde dichas posiciones tienen probabilidad = 0.

De esta manera matemática, la máscara bloquea toda mirada a tokens futuros.

Tabla4.

Tabla con máscara

	The	El	Niño	phenomenon
The	2.1	$-\infty$	$-\infty$	$-\infty$
El	1.2	3.5	$-\infty$	$-\infty$
Niño	0.9	1.7	4.1	$-\infty$
phenomenon	1.4	2.1	2.7	4.5

Si quisiéramos pensar en Python, lo descrito anteriormente

```
python
scores = Q @ K.transpose(-2, -1)

scores = scores / math.sqrt(d_k)

scores = scores.masked_fill(
```

```

        mask == 0,
        float("-inf")
    )

    attention = torch.softmax(
        scores,
        dim=-1
    )

```

En un supuesto escenario de trabajo del Decoder, los 1 significan que se puede prestar atención y lo ceros que no.

Tabla 5.

Tabla con máscara triangular

	The	El	Niño	phenomenon
The	1	0	0	0
El	1	1	0	0
Niño	1	1	1	0
phenomenon	1	1	1	1

Para cerra esta parte, la idea queda como:

Masked Self-Attention = Self-Attention + una máscara causal que impide que cada posición vea información futura.

Y esa pequeña modificación es la que permite que un Decoder pueda generar texto de manera autoregresiva, token por token, sin conocer de antemano la respuesta completa.

3.17 Cross-Attention

Cross-Attention es la fase donde el Decoder consulta al Encoder: El Decoder tiene "The El Niño ..." y necesita decidir qué información de la entrada original debe utilizar.

Aquí:

Query → Decoder

Key → Encoder

Value → Encoder

El decoder puede prestar más atención a:

fenómeno

El

Niño

para decidir qué palabra generar.

Luego se procede a calcular probabilidades para el siguiente token por ejemplo

phenomenon 0.72

event 0.08

storm 0.04

weather 0.03

...

Por tanto, se selecciona **phenomenon** y ahora la nueva secuencia es:

The El Niño phenomenon

Y de esta manera el proceso continuo de manera sucesiva

Ahora la pregunta tal vez sea ¿Cómo decide el siguiente token?.

Al final del Decoder ha y un vector ;

$h \in \mathbb{R}^{d_{model}}$, donde la capa Linear lo transforma $z = hW + b$

Si tenemos un vocabulario de 25 000 tokens: $z \in \mathbb{R}^{25000}$

Entonces tenemos un valor para cada token, por ejemplo:

The	-1.4
phenomenon	5.8

```

storm          2.1
is             4.9
rainfall      1.3
...

```

Después se aplica Softmax para obtener probabilidades, pues Softmax se encarga de convertir números en probabilidades.

$$P(\text{token}_i) = \text{Softmax}(z_i)$$

```

phenomenon → 72 %
event      → 8 %
storm      → 4 %
weather    → 3 %
...

```

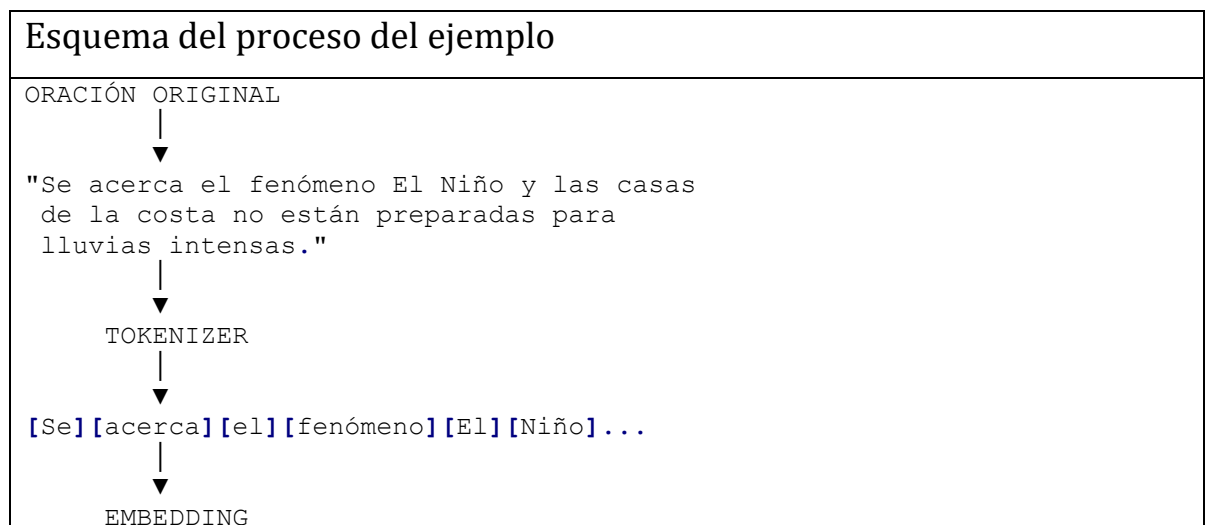
Y el modelo puede seleccionar una de las probabilidades.

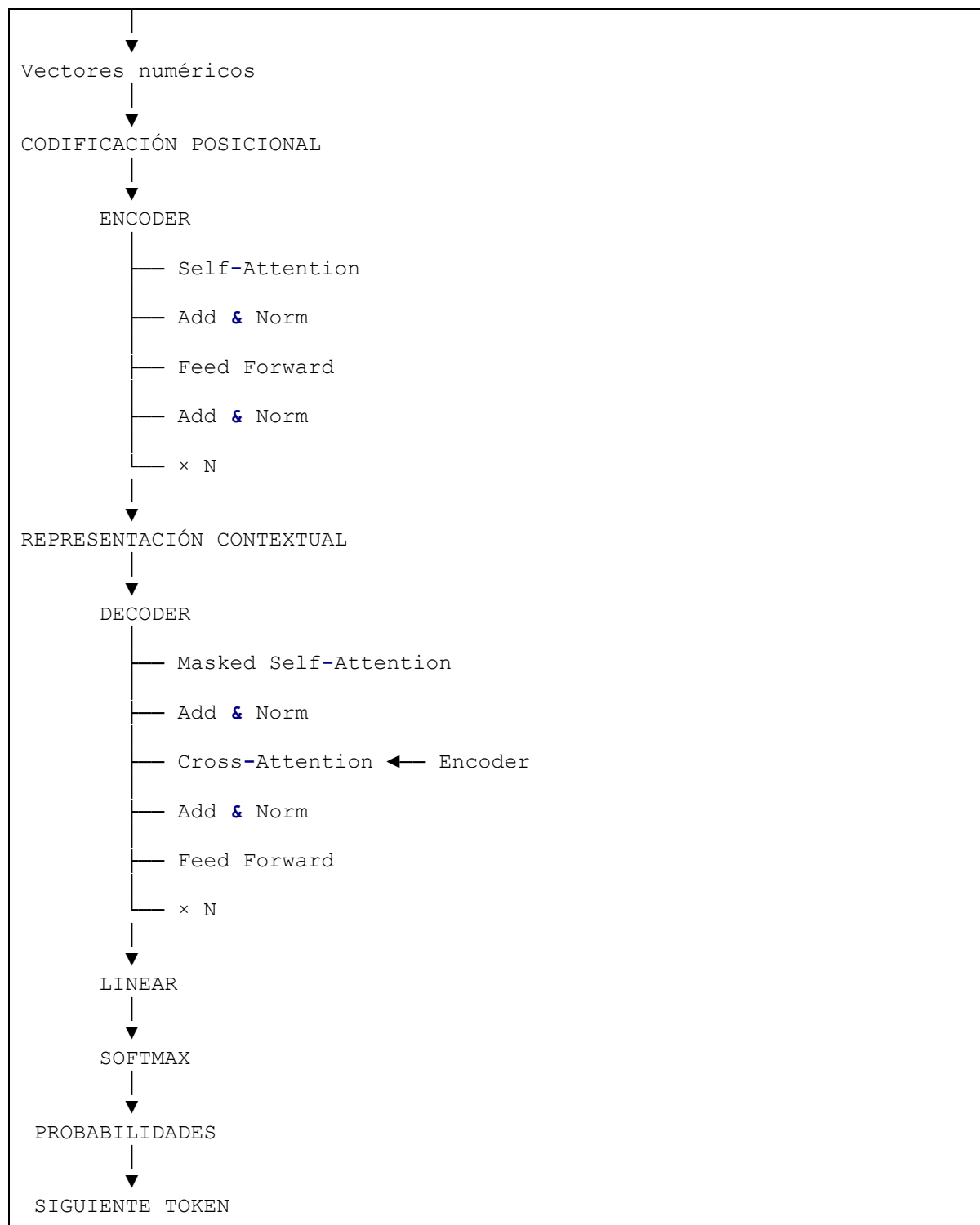
```

Se tendría un proceso iterativo
probabilidades
↓
token seleccionado
↓
se agrega a la secuencia
↓
el decoder vuelve a ejecutarse

```

3.18 Estructura muy completa de todo lo descrito





3.19 Como aprende el transformer

Supongamos que durante el entrenamiento tenemos como entrada:

Se acerca el fenómeno El Niño... y el siguiente token correcto es el token “y”,

El modelo podría predecir:

y → 0.20
las → 0.30
porque → 0.10
...

La respuesta correcta es “y”. Entonces se calcula una función de pérdida, por ejemplo cross-entropy. El error se propaga hacia atrás con Loss, Softmax, Linear, Decoder, Attention, Embedding mediante backpropagation.

Después un optimizador modifica los pesos: $W \leftarrow W - \text{learning_rate} \times \text{gradient}$. El optimizador puede ser SGD, Adam, AdamW, etc. Y este proceso se repite miles, diría millones de veces.

Para culminar esta parte: La verdadera fuerza del Transformer está en Self-Attention + Multi-Head Attention. Pues el modelo puede establecer relaciones entre múltiples posiciones de la secuencia y construir una representación del contexto. En términos sencillos “junior”: el Encoder construye el contexto; el Decoder utiliza ese contexto para decidir, paso a paso, qué token debe predecir después.

3.20 Código del DECODER para replicar

Código ilustrativo y reproducible del Decoder

- Entrada objetivo del Decoder.
- Embedding.
- Positional Encoding.
- Masked Self-Attention.
- Máscara causal.
- Add & Norm.
- Cross-Attention.
- Relación Decoder → Encoder.

- Add & Norm.
- Feed Forward.
- Add & Norm.
- Salida final del Decoder.
- Linear.
- Softmax.
- Generación autoregresiva token por token.

Decoder.py

```
# =====
# TRANSFORMER - DECODER
# =====
#
# Texto del Encoder:
#
# "Se acerca el fenómeno El Niño y las casas de la costa
# no están preparadas para lluvias intensas"
#
# Traducción objetivo:
#
# "The El Niño phenomenon is approaching and the coastal
# houses are not prepared for heavy rainfall."
#
# =====
#
# IMPORTANTE:
#
# Este programa es una DEMOSTRACIÓN PEDAGÓGICA.
#
# Los pesos de las capas neuronales son aleatorios.
# Por tanto, el modelo NO ha aprendido inglés ni traducción.
#
# Para mostrar la generación autoregresiva coherentemente,
# utilizaremos la traducción objetivo como referencia
# pedagógica.
#
# Esto NO debe interpretarse como una traducción producida
# por un modelo entrenado.
#
# =====

import math
import re
import torch
import torch.nn as nn
import torch.nn.functional as F

# =====
# 1. CONFIGURACIÓN
# =====

SEED = 42
```

```

torch.manual_seed(SEED)

print("=" * 80)
print("TRANSFORMER - DECODER PASO A PASO")
print("=" * 80)

print("\nSeed:", SEED)

# Dimensiones
d_model = 32
num_heads = 4
d_ff = 64

# =====
# 2. TEXTO DEL ENCODER
# =====

texto_encoder = (
    "Se acerca el fenómeno El Niño y las casas de la costa "
    "no están preparadas para lluvias intensas"
)

# Traducción objetivo
texto_objetivo = (
    "The El Niño phenomenon is approaching and the coastal "
    "houses are not prepared for heavy rainfall"
)

print("\n" + "=" * 80)
print("FASE 1 - INFORMACIÓN DEL ENCODER Y OBJETIVO DEL DECODER")
print("=" * 80)

print("\nEntrada del Encoder:")
print(texto_encoder)

print("\nTraducción objetivo:")
print(texto_objetivo)

# =====
# 3. TOKENIZADOR SIMPLE
# =====

def tokenizar(texto):

    return re.findall(
        r"\w+|[\^\w\s]",
        texto,
        flags=re.UNICODE
    )

tokens_encoder = tokenizar(
    texto_encoder
)

tokens_target = tokenizar(
    texto_objetivo
)

```

```

print("\nTokens del Encoder:")
print(tokens_encoder)

print("\nTokens objetivo:")
print(tokens_target)

# =====
# 4. CREAR VOCABULARIO
# =====
#
# En una aplicación real se utilizaría un tokenizer como
# BPE, WordPiece o SentencePiece.
#
# Aquí utilizamos un tokenizer simple para poder observar
# cada operación.
# =====

special_tokens = [
    "<PAD>",
    "<UNK>",
    "<BOS>",
    "<EOS>"
]

todos_los_tokens = (
    special_tokens
    + tokens_encoder
    + tokens_target
)

# Eliminar duplicados conservando orden
vocab = list(
    dict.fromkeys(todos_los_tokens)
)

stoi = {
    token: i
    for i, token in enumerate(vocab)
}

itos = {
    i: token
    for token, i in stoi.items()
}

vocab_size = len(vocab)

print("\n" + "=" * 80)
print("FASE 2 - VOCABULARIO")
print("=" * 80)

print("\nTamaño del vocabulario:", vocab_size)

for i, token in enumerate(vocab):
    print(

```

```

        f"{i:3d} -> {token}"
    )

# =====
# 5. IDS DEL ENCODER
# =====

encoder_ids = torch.tensor(
    [
        [
            stoi[token]
            for token in tokens_encoder
        ]
    ],
    dtype=torch.long
)

print("\n" + "=" * 80)
print("FASE 3 - REPRESENTACIÓN DEL ENCODER")
print("=" * 80)

print("\nEncoder IDs:")
print(encoder_ids)

print("\nShape:")
print(encoder_ids.shape)

# =====
# 6. EMBEDDING
# =====

embedding = nn.Embedding(
    vocab_size,
    d_model
)

encoder_embeddings = embedding(
    encoder_ids
)

print("\nEmbeddings del Encoder:")
print(
    encoder_embeddings.shape
)

# =====
# 7. POSITIONAL ENCODING
# =====

def positional_encoding(
    seq_len,
    d_model
):
    pe = torch.zeros(
        seq_len,
        d_model
    )

```

```

    position = torch.arange(
        0,
        seq_len,
        dtype=torch.float
    ).unsqueeze(1)

    div_term = torch.exp(
        torch.arange(
            0,
            d_model,
            2,
            dtype=torch.float
        )
        * (-math.log(10000.0) / d_model)
    )

    pe[:, 0::2] = torch.sin(
        position * div_term
    )

    pe[:, 1::2] = torch.cos(
        position * div_term
    )

    return pe

pe_encoder = positional_encoding(
    len(tokens_encoder),
    d_model
)

encoder_output = (
    encoder_embeddings
    + pe_encoder.unsqueeze(0)
)

# =====
# 8. FINAL DEL ENCODER
# =====

print("\n" + "=" * 80)
print("FASE 4 - SALIDA FINAL DEL ENCODER")
print("=" * 80)

print("\nEncoder Output:")
print(
    encoder_output.shape
)

print("\nPrimer token - primeros 8 valores:")

print(
    [
        round(float(x), 4)
        for x in encoder_output[
            0, 0, :8
        ]
    ]
)

```

```

print("\nÚltimo token - primeros 8 valores:")

print(
    [
        round(float(x), 4)
        for x in encoder_output[
            0, -1, :8
        ]
    ]
)

# =====
# 9. ENTRADA INICIAL DEL DECODER
# =====
#
# El Decoder comienza con:
#
# <BOS>
#
# BOS = Beginning Of Sequence
#
# En generación autoregresiva:
#
# <BOS>
#   ↓
# The
#   ↓
# The El
#   ↓
# The El Niño
#   ↓
# ...
#
# =====

decoder_tokens = [
    "<BOS>"
]

print("\n" + "=" * 80)
print("FASE 5 - INICIO DEL DECODER")
print("=" * 80)

print("\nEntrada inicial:")
print(decoder_tokens)

# =====
# 10. CLASE MULTI-HEAD ATTENTION
# =====

class MultiHeadAttention(
    nn.Module
):
    def __init__(
        self,
        d_model,
        num_heads
    ):

```



```

super().__init__()

assert (
    d_model % num_heads == 0
)

self.d_model = d_model
self.num_heads = num_heads
self.d_k = d_model // num_heads

self.W_Q = nn.Linear(
    d_model,
    d_model
)

self.W_K = nn.Linear(
    d_model,
    d_model
)

self.W_V = nn.Linear(
    d_model,
    d_model
)

self.W_O = nn.Linear(
    d_model,
    d_model
)

self.attention_weights = None

def split_heads(self, x):

    batch, seq_len, _ = x.shape

    x = x.view(
        batch,
        seq_len,
        self.num_heads,
        self.d_k
    )

    return x.transpose(1, 2)

def combine_heads(self, x):

    batch, heads, seq_len, d_k = x.shape

    x = x.transpose(1, 2)

    return x.contiguous().view(
        batch,
        seq_len,
        self.d_model
    )

def forward(
    self,
    query,
    key,
    value,
    mask=None
):

```

```

# -----
# Q K V
# -----

Q = self.W_Q(query)

K = self.W_K(key)

V = self.W_V(value)

# -----
# Dividir en cabezas
# -----

Q = self.split_heads(Q)

K = self.split_heads(K)

V = self.split_heads(V)

# -----
# Scaled Dot Product Attention
# -----

scores = torch.matmul(
    Q,
    K.transpose(-2, -1)
)

scores = (
    scores
    / math.sqrt(self.d_k)
)

# -----
# MASK
# -----

if mask is not None:

    scores = scores.masked_fill(
        mask == 0,
        float("-inf")
    )

# -----
# SOFTMAX
# -----

attention = torch.softmax(
    scores,
    dim=-1
)

self.attention_weights = (
    attention.detach()
)

# -----
# Attention × V
# -----

context = torch.matmul(

```

```

        attention,
        v
    )

    # -----
    # Combinar heads
    # -----

    context = self.combine_heads(
        context
    )

    # -----
    # W_O
    # -----

    output = self.W_O(
        context
    )

    return output

# =====
# 11. FUNCIÓN PARA CREAR MÁSCARA CAUSAL
# =====

def crear_mascara_causal(
    longitud
):
    mask = torch.tril(
        torch.ones(
            longitud,
            longitud
        )
    )

    return mask.unsqueeze(0).unsqueeze(0)

# =====
# 12. FUNCIÓN ADD & NORM
# =====

class AddNorm(nn.Module):
    def __init__(
        self,
        d_model
    ):
        super().__init__()

        self.norm = nn.LayerNorm(
            d_model
        )

    def forward(
        self,
        x,
        sublayer_output
    ):

```

```

        return self.norm(
            x + sublayer_output
        )

# =====
# 13. FEED FORWARD
# =====

class FeedForward(
    nn.Module
):

    def __init__(
        self,
        d_model,
        d_ff
    ):

        super().__init__()

        self.linear1 = nn.Linear(
            d_model,
            d_ff
        )

        self.linear2 = nn.Linear(
            d_ff,
            d_model
        )

    def forward(self, x):

        x = self.linear1(x)

        x = F.gelu(x)

        x = self.linear2(x)

        return x

# =====
# 14. INSTANCIAR COMPONENTES DEL DECODER
# =====

masked_self_attention = (
    MultiHeadAttention(
        d_model,
        num_heads
    )
)

cross_attention = (
    MultiHeadAttention(
        d_model,
        num_heads
    )
)

add_norm_1 = AddNorm(
    d_model

```

```

)

add_norm_2 = AddNorm(
    d_model
)

add_norm_3 = AddNorm(
    d_model
)

feed_forward = FeedForward(
    d_model,
    d_ff
)

output_projection = nn.Linear(
    d_model,
    vocab_size
)

# =====
# 15. FUNCIÓN QUE EJECUTA EL DECODER
# =====

def ejecutar_decoder(
    decoder_tokens,
    mostrar_detalle=True
):
    # -----
    # Convertir tokens a IDs
    # -----

    decoder_ids = torch.tensor(
        [
            [
                stoi[token]
                for token in decoder_tokens
            ]
        ],
        dtype=torch.long
    )

    # -----
    # EMBEDDING
    # -----

    decoder_embeddings = embedding(
        decoder_ids
    )

    # -----
    # POSITIONAL ENCODING
    # -----

    pe = positional_encoding(
        len(decoder_tokens),
        d_model
    )

```

```

decoder_x = (
    decoder_embeddings
    + pe.unsqueez(0)
)

# -----
# MASKED SELF-ATTENTION
# -----

mask = crear_mascara_causal(
    len(decoder_tokens)
)

self_attention_output = (
    masked_self_attention(
        decoder_x,
        decoder_x,
        decoder_x,
        mask
    )
)

# -----
# ADD & NORM 1
# -----

decoder_x1 = add_norm_1(
    decoder_x,
    self_attention_output
)

# -----
# CROSS-ATTENTION
#
# Q = Decoder
# K = Encoder
# V = Encoder
# -----

cross_attention_output = (
    cross_attention(
        decoder_x1,
        encoder_output,
        encoder_output
    )
)

# -----
# ADD & NORM 2
# -----

decoder_x2 = add_norm_2(
    decoder_x1,
    cross_attention_output
)

# -----
# FEED FORWARD
# -----

ffn_output = feed_forward(
    decoder_x2
)

```

```

# -----
# ADD & NORM 3
# -----

decoder_output = add_norm_3(
    decoder_x2,
    ffn_output
)

# -----
# LINEAR
# -----

logits = output_projection(
    decoder_output
)

# -----
# SOFTMAX
# -----

probabilities = torch.softmax(
    logits,
    dim=-1
)

if mostrar_detalle:

    print("\n" + "-" * 80)

    print(
        "TOKENS ACTUALES DEL DECODER:"
    )

    print(
        decoder_tokens
    )

    print(
        "\nDecoder IDs:"
    )

    print(
        decoder_ids
    )

    print(
        "\nEmbedding:"
    )

    print(
        decoder_embeddings.shape
    )

    print(
        "\nPositional Encoding:"
    )

    print(
        decoder_x.shape
    )

    print(

```

```

        "\nMáscara causal:"
    )

    print(
        mask[0, 0]
    )

    print(
        "\nMasked Self-Attention:"
    )

    print(
        self_attention_output.shape
    )

    print(
        "\nAdd & Norm 1:"
    )

    print(
        decoder_x1.shape
    )

    print(
        "\nCross-Attention:"
    )

    print(
        cross_attention_output.shape
    )

    print(
        "\nAdd & Norm 2:"
    )

    print(
        decoder_x2.shape
    )

    print(
        "\nFeed Forward:"
    )

    print(
        f"{d_model} -> {d_ff} -> {d_model}"
    )

    print(
        "\nSalida FFN:"
    )

    print(
        ffn_output.shape
    )

    print(
        "\nAdd & Norm 3:"
    )

    print(
        decoder_output.shape
    )

    print(

```



```

        "\nLogits:"
    )

    print(
        logits.shape
    )

    print(
        "\nProbabilidades:"
    )

    print(
        probabilities.shape
    )

    return probabilities

# =====
# 16. GENERACIÓN AUTOREGRESIVA
# =====
#
# AQUÍ DEMOSTRAMOS EL PROCESO:
#
# <BOS>
#   ↓
# The
#
# <BOS> The
#   ↓
# El
#
# <BOS> The El
#   ↓
# Niño
#
# ...
#
# =====

print("\n\n")
print("=" * 80)
print("FASE 6 - GENERACIÓN AUTOREGRESIVA")
print("=" * 80)

# -----
# Secuencia objetivo de referencia
# -----

target_sequence = (
    ["<BOS>"]
    + tokens_target
    + ["<EOS>"]
)

generated_tokens = [
    "<BOS>"
]

# -----
# Mostrar objetivo

```

```

# -----

print("\nSecuencia objetivo de referencia:")

print(
    " ".join(target_sequence)
)

print("\n")
print(
    "NOTA: La siguiente generación es pedagógica."
)

print(
    "El siguiente token se seleccionará de la "
    "secuencia objetivo para demostrar el mecanismo "
    "autoregresivo."
)

# =====
# 17. ITERACIONES
# =====

for paso in range(
    1,
    len(target_sequence)
):

    print("\n")
    print("#" * 80)

    print(
        f"PASO AUTOREGRESIVO {paso}"
    )

    print("#" * 80)

    print(
        "\nEntrada actual del Decoder:"
    )

    print(
        " ".join(generated_tokens)
    )

    # -----
    # Ejecutar todo el Decoder
    # -----

    probabilities = ejecutar_decoder(
        generated_tokens,
        mostrar_detalle=True
    )

    # -----
    # Obtener la distribución de probabilidad real
    # producida por las capas actuales.
    # -----

```

```

ultima_posicion = (
    len(generated_tokens) - 1
)

ultima_probabilidades = (
    probabilities[
        0,
        ultima_posicion
    ]
)

top_probs, top_ids = torch.topk(
    ultima_probabilidades,
    k=min(5, vocab_size)
)

print(
    "\nTop-5 predicciones del "
    "Decoder con pesos aleatorios:"
)

for rank, (
    prob,
    token_id
) in enumerate(
    zip(top_probs, top_ids),
    start=1
):

    token = itos[
        int(token_id)
    ]

    print(
        f"{rank}. "
        f"{token:15s} "
        f"{float(prob):.4f}"
    )

# -----
# TOKEN OBJETIVO
# -----
#
# Para la demostración utilizamos el siguiente token
# de la traducción objetivo.
#
# Esto es equivalente a una demostración controlada
# del proceso, NO a una predicción aprendida.
# -----

siguiente_token = (
    target_sequence[paso]
)

print(
    "\nSiguiente token de referencia:"
)

```

```

print(
    siguiente_token
)

generated_tokens.append(
    siguiente_token
)

print(
    "\nSecuencia generada hasta ahora:"
)

print(
    " ".join(generated_tokens)
)

# -----
# EOS
# -----

if siguiente_token == "<EOS>":

    print(
        "\nSe recibió <EOS>."
    )

    print(
        "La generación ha terminado."
    )

    break

# =====
# 18. RESULTADO FINAL
# =====

print("\n\n")
print("=" * 80)
print("FASE 7 - RESULTADO FINAL DEL DECODER")
print("=" * 80)

print(
    "\nSecuencia generada:"
)

print(
    " ".join(generated_tokens)
)

# Eliminar tokens especiales
resultado = [
    token
    for token in generated_tokens
    if token not in [
        "<BOS>",
        "<EOS>"
    ]
]

```

```

print(
    "\nTraducción final:"
)

print(
    " ".join(resultado)
)

# =====
# 19. COMPROBAR CONTRA LA TRADUCCIÓN OBJETIVO
# =====

print("\n" + "=" * 80)
print("FASE 8 - COMPARACIÓN CON LA REFERENCIA")
print("=" * 80)

print(
    "\nObjetivo:"
)

print(
    texto_objetivo
)

print(
    "\nResultado pedagógico:"
)

print(
    " ".join(resultado)
)

if (
    " ".join(resultado)
    == texto_objetivo
):

    print(
        "\nResultado: coincide con la "
        "secuencia objetivo de referencia."
    )

else:

    print(
        "\nResultado: diferente de la referencia."
    )

# =====
# 20. RESUMEN FINAL
# =====

print("\n" + "=" * 80)
print("RESUMEN DEL PROCESO")
print("=" * 80)

print(

```

```

    """
ENCODER
|
| Encoder Output
|
v
DECODER
|
+--> Embedding
|
+--> Positional Encoding
|
+--> Masked Self-Attention
|
+--> Add & Norm
|
+--> Cross-Attention
|           |
|           +--> Q = Decoder
|           +--> K = Encoder
|           +--> V = Encoder
|
+--> Add & Norm
|
+--> Feed Forward
|
+--> Add & Norm
|
v
DECODER OUTPUT
|
v
LINEAR
|
v
LOGITS
|
v
SOFTMAX
|
v
PROBABILIDADES
|
v
SIGUIENTE TOKEN
|
v
SE INCORPORA AL DECODER
|
+-----+
|
|
v
SIGUIENTE PASO

    """
)

print("=" * 80)
print("FIN DEL PROGRAMA")
print("=" * 80)

```

Salida

Add & Norm 2:
torch.Size([1, 5, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 5, 32])

Add & Norm 3:
torch.Size([1, 5, 32])

Logits:
torch.Size([1, 5, 36])

Probabilidades:
torch.Size([1, 5, 36])

Top-5 predicciones del Decoder con pesos aleatorios:

1. Se	0.0742
2. approaching	0.0636
3. The	0.0538
4. rainfall	0.0515
5. <PAD>	0.0407

Siguiente token de referencia:
is

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is

```
#####  
##  
PASO AUTOREGRESIVO 6  
#####  
##
```

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is

--
TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is']

Decoder IDs:
tensor([[2, 22, 8, 9, 23, 24]])

Embedding:
torch.Size([1, 6, 32])

Positional Encoding:
torch.Size([1, 6, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0.],
[1., 1., 0., 0., 0., 0.],
[1., 1., 1., 0., 0., 0.],
[1., 1., 1., 1., 0., 0.],
[1., 1., 1., 1., 1., 0.]])

```

[1., 1., 1., 1., 1., 1.]]

Masked Self-Attention:
torch.Size([1, 6, 32])

Add & Norm 1:
torch.Size([1, 6, 32])

Cross-Attention:
torch.Size([1, 6, 32])

Add & Norm 2:
torch.Size([1, 6, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 6, 32])

Add & Norm 3:
torch.Size([1, 6, 32])

Logits:
torch.Size([1, 6, 36])

Probabilidades:
torch.Size([1, 6, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. Niño          0.0792
2. Se            0.0723
3. <UNK>         0.0635
4. fenómeno     0.0462
5. approaching  0.0439

Siguiente token de referencia:
approaching

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching

#####
##
PASO AUTOREGRESIVO 7
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching

-----
--

TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25]])

Embedding:
torch.Size([1, 7, 32])

Positional Encoding:

```



```

torch.Size([1, 7, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 1., 0.],
        [1., 1., 1., 1., 1., 1., 1.]])

Masked Self-Attention:
torch.Size([1, 7, 32])

Add & Norm 1:
torch.Size([1, 7, 32])

Cross-Attention:
torch.Size([1, 7, 32])

Add & Norm 2:
torch.Size([1, 7, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 7, 32])

Add & Norm 3:
torch.Size([1, 7, 32])

Logits:
torch.Size([1, 7, 36])

Probabilidades:
torch.Size([1, 7, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. intensas      0.0717
2. <UNK>         0.0638
3. no           0.0597
4. Se           0.0574
5. coastal      0.0537

Siguiente token de referencia:
and

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching and

#####
##
PASO AUTOREGRESIVO 8
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching and

-----
--
TOKENS ACTUALES DEL DECODER:

```

```

['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26]])

Embedding:
torch.Size([1, 8, 32])

Positional Encoding:
torch.Size([1, 8, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1.]])

Masked Self-Attention:
torch.Size([1, 8, 32])

Add & Norm 1:
torch.Size([1, 8, 32])

Cross-Attention:
torch.Size([1, 8, 32])

Add & Norm 2:
torch.Size([1, 8, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 8, 32])

Add & Norm 3:
torch.Size([1, 8, 32])

Logits:
torch.Size([1, 8, 36])

Probabilidades:
torch.Size([1, 8, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. preparadas      0.0740
2. coastal         0.0711
3. el              0.0558
4. no              0.0499
5. approaching     0.0470

Siguiente token de referencia:
the

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching and the

#####
##

```

```

PASO AUTOREGRESIVO 9
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching and the

-----
--
TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',
'the']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27]])

Embedding:
torch.Size([1, 9, 32])

Positional Encoding:
torch.Size([1, 9, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1.]])

Masked Self-Attention:
torch.Size([1, 9, 32])

Add & Norm 1:
torch.Size([1, 9, 32])

Cross-Attention:
torch.Size([1, 9, 32])

Add & Norm 2:
torch.Size([1, 9, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 9, 32])

Add & Norm 3:
torch.Size([1, 9, 32])

Logits:
torch.Size([1, 9, 36])

Probabilidades:
torch.Size([1, 9, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. <UNK>          0.0737
2. de             0.0671
3. preparadas     0.0640

```

```
4. are 0.0477
5. el 0.0418
```

Siguiente token de referencia:
coastal

Secuencia generada hasta ahora:

<BOS> The El Niño phenomenon **is** approaching **and** the coastal

```
#####
##
PASO AUTOREGRESIVO 10
#####
##
```

Entrada actual **del** Decoder:

<BOS> The El Niño phenomenon **is** approaching **and** the coastal

```
-----
--
```

TOKENS ACTUALES DEL DECODER:

```
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',
'the', 'coastal']
```

Decoder IDs:

```
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27, 28]])
```

Embedding:

```
torch.Size([1, 10, 32])
```

Positional Encoding:

```
torch.Size([1, 10, 32])
```

Máscara causal:

```
tensor([[[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
         [1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
         [1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
         [1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
         [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
         [1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
         [1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
         [1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
         [1., 1., 1., 1., 1., 1., 1., 1., 1., 0.],
         [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

Masked Self-Attention:

```
torch.Size([1, 10, 32])
```

Add & Norm 1:

```
torch.Size([1, 10, 32])
```

Cross-Attention:

```
torch.Size([1, 10, 32])
```

Add & Norm 2:

```
torch.Size([1, 10, 32])
```

Feed Forward:

```
32 -> 64 -> 32
```

Salida FFN:

```
torch.Size([1, 10, 32])
```

```

Add & Norm 3:
torch.Size([1, 10, 32])

Logits:
torch.Size([1, 10, 36])

Probabilidades:
torch.Size([1, 10, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. approaching      0.0694
2. El                0.0636
3. Niño             0.0601
4. están            0.0589
5. no                0.0566

Siguiente token de referencia:
houses

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching and the coastal houses

#####
##
PASO AUTOREGRESIVO 11
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching and the coastal houses

-----
--
TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',
'the', 'coastal', 'houses']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27, 28, 29]])

Embedding:
torch.Size([1, 11, 32])

Positional Encoding:
torch.Size([1, 11, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])

Masked Self-Attention:
torch.Size([1, 11, 32])

Add & Norm 1:

```

```

torch.Size([1, 11, 32])

Cross-Attention:
torch.Size([1, 11, 32])

Add & Norm 2:
torch.Size([1, 11, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 11, 32])

Add & Norm 3:
torch.Size([1, 11, 32])

Logits:
torch.Size([1, 11, 36])

Probabilidades:
torch.Size([1, 11, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. preparadas      0.0779
2. <UNK>            0.0711
3. Niño            0.0663
4. intensas        0.0565
5. el              0.0463

Siguiente token de referencia:
are

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching and the coastal houses are

#####
##
PASO AUTOREGRESIVO 12
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching and the coastal houses are

-----
--
TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',
'the', 'coastal', 'houses', 'are']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27, 28, 29, 30]])

Embedding:
torch.Size([1, 12, 32])

Positional Encoding:
torch.Size([1, 12, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])

```

```
[1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

Masked Self-Attention:

```
torch.Size([1, 12, 32])
```

Add & Norm 1:

```
torch.Size([1, 12, 32])
```

Cross-Attention:

```
torch.Size([1, 12, 32])
```

Add & Norm 2:

```
torch.Size([1, 12, 32])
```

Feed Forward:

```
32 -> 64 -> 32
```

Salida FFN:

```
torch.Size([1, 12, 32])
```

Add & Norm 3:

```
torch.Size([1, 12, 32])
```

Logits:

```
torch.Size([1, 12, 36])
```

Probabilidades:

```
torch.Size([1, 12, 36])
```

Top-5 predicciones **del** Decoder con pesos aleatorios:

```
1. not          0.0768
2. no           0.0568
3. <PAD>        0.0539
4. intensas    0.0524
5. están       0.0510
```

Siguiente token de referencia:

```
not
```

Secuencia generada hasta ahora:

```
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
```

```
#####
##
PASO AUTOREGRESIVO 13
#####
##
```

Entrada actual **del** Decoder:

```
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
```

```
-----
--
```

TOKENS ACTUALES DEL DECODER:

```
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',  
'the', 'coastal', 'houses', 'are', 'not']
```

Decoder IDs:

```
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27, 28, 29, 30, 31]])
```

Embedding:

```
torch.Size([1, 13, 32])
```

Positional Encoding:

```
torch.Size([1, 13, 32])
```

Máscara causal:

```
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],  
        [1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],  
        [1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],  
        [1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],  
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],  
        [1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],  
        [1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],  
        [1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],  
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],  
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],  
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],  
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0.],  
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

Masked Self-Attention:

```
torch.Size([1, 13, 32])
```

Add & Norm 1:

```
torch.Size([1, 13, 32])
```

Cross-Attention:

```
torch.Size([1, 13, 32])
```

Add & Norm 2:

```
torch.Size([1, 13, 32])
```

Feed Forward:

```
32 -> 64 -> 32
```

Salida FFN:

```
torch.Size([1, 13, 32])
```

Add & Norm 3:

```
torch.Size([1, 13, 32])
```

Logits:

```
torch.Size([1, 13, 36])
```

Probabilidades:

```
torch.Size([1, 13, 36])
```

Top-5 predicciones **del** Decoder con pesos aleatorios:

```
1. preparadas    0.1304  
2. coastal       0.0725  
3. están         0.0588  
4. no            0.0489  
5. the           0.0405
```

Siguiente token de referencia:

```
prepared
```


Secuencia generada hasta ahora:

<BOS> The El Niño phenomenon **is** approaching **and** the coastal houses are **not** prepared

##

PASO AUTOREGRESIVO 14

##

Entrada actual **del** Decoder:

<BOS> The El Niño phenomenon **is** approaching **and** the coastal houses are **not** prepared

--

TOKENS ACTUALES DEL DECODER:

['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and', 'the', 'coastal', 'houses', 'are', 'not', 'prepared']

Decoder IDs:

tensor([[2, 22, 8, 9, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32]])

Embedding:

torch.Size([1, 14, 32])

Positional Encoding:

torch.Size([1, 14, 32])

Máscara causal:

tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0.],
[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])

Masked Self-Attention:

torch.Size([1, 14, 32])

Add & Norm 1:

torch.Size([1, 14, 32])

Cross-Attention:

torch.Size([1, 14, 32])

Add & Norm 2:

torch.Size([1, 14, 32])

Feed Forward:

32 -> 64 -> 32

Salida FFN:

torch.Size([1, 14, 32])

```

Add & Norm 3:
torch.Size([1, 14, 32])

Logits:
torch.Size([1, 14, 36])

Probabilidades:
torch.Size([1, 14, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. la 0.1107
2. preparadas 0.0831
3. coastal 0.0526
4. not 0.0424
5. intensas 0.0417

Siguiente token de referencia:
for

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
prepared for

#####
##
PASO AUTOREGRESIVO 15
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
prepared for

-----
--
TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',
'the', 'coastal', 'houses', 'are', 'not', 'prepared', 'for']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]])

Embedding:
torch.Size([1, 15, 32])

Positional Encoding:
torch.Size([1, 15, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0.]])

```

```

[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]]))

Masked Self-Attention:
torch.Size([1, 15, 32])

Add & Norm 1:
torch.Size([1, 15, 32])

Cross-Attention:
torch.Size([1, 15, 32])

Add & Norm 2:
torch.Size([1, 15, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 15, 32])

Add & Norm 3:
torch.Size([1, 15, 32])

Logits:
torch.Size([1, 15, 36])

Probabilidades:
torch.Size([1, 15, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. <BOS>          0.0569
2. fenómeno      0.0542
3. <EOS>          0.0489
4. The           0.0479
5. approaching   0.0440

Siguiente token de referencia:
heavy

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
prepared for heavy

#####
##
PASO AUTOREGRESIVO 16
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
prepared for heavy

-----
--
TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',
'the', 'coastal', 'houses', 'are', 'not', 'prepared', 'for', 'heavy']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34]])

Embedding:

```

```

torch.Size([1, 16, 32])

Positional Encoding:
torch.Size([1, 16, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])

Masked Self-Attention:
torch.Size([1, 16, 32])

Add & Norm 1:
torch.Size([1, 16, 32])

Cross-Attention:
torch.Size([1, 16, 32])

Add & Norm 2:
torch.Size([1, 16, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 16, 32])

Add & Norm 3:
torch.Size([1, 16, 32])

Logits:
torch.Size([1, 16, 36])

Probabilidades:
torch.Size([1, 16, 36])

Top-5 predicciones del Decoder con pesos aleatorios:
1. no 0.1044
2. rainfall 0.0544
3. coastal 0.0543
4. <PAD> 0.0526
5. acerca 0.0453

Siguiente token de referencia:
rainfall

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
prepared for heavy rainfall

```

```
#####
##
PASO AUTOREGRESIVO 17
#####
##

Entrada actual del Decoder:
<BOS> The El Niño phenomenon is approaching and the coastal houses are not
prepared for heavy rainfall

-----
--
TOKENS ACTUALES DEL DECODER:
['<BOS>', 'The', 'El', 'Niño', 'phenomenon', 'is', 'approaching', 'and',
'the', 'coastal', 'houses', 'are', 'not', 'prepared', 'for', 'heavy',
'rainfall']

Decoder IDs:
tensor([[ 2, 22,  8,  9, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35]])

Embedding:
torch.Size([1, 17, 32])

Positional Encoding:
torch.Size([1, 17, 32])

Máscara causal:
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])

Masked Self-Attention:
torch.Size([1, 17, 32])

Add & Norm 1:
torch.Size([1, 17, 32])

Cross-Attention:
torch.Size([1, 17, 32])

Add & Norm 2:
torch.Size([1, 17, 32])

Feed Forward:
32 -> 64 -> 32

Salida FFN:
torch.Size([1, 17, 32])
```

Add & Norm 3:
torch.Size([1, 17, 32])

Logits:
torch.Size([1, 17, 36])

Probabilidades:
torch.Size([1, 17, 36])

Top-5 predicciones del Decoder con pesos aleatorios:

1. lluvias	0.0817
2. rainfall	0.0665
3. no	0.0647
4. <UNK>	0.0548
5. de	0.0524

Siguiente token de referencia:
<EOS>

Secuencia generada hasta ahora:
<BOS> The El Niño phenomenon **is** approaching **and** the coastal houses are **not** prepared **for** heavy rainfall <EOS>

Se recibió <EOS>.
La generación ha terminado.

=====

==

FASE 7 - RESULTADO FINAL DEL DECODER

=====

==

Secuencia generada:
<BOS> The El Niño phenomenon **is** approaching **and** the coastal houses are **not** prepared **for** heavy rainfall <EOS>

Traducción final:
The El Niño phenomenon **is** approaching **and** the coastal houses are **not** prepared **for** heavy rainfall

=====

==

FASE 8 - COMPARACIÓN CON LA REFERENCIA

=====

==

Objetivo:
The El Niño phenomenon **is** approaching **and** the coastal houses are **not** prepared **for** heavy rainfall

Resultado pedagógico:
The El Niño phenomenon **is** approaching **and** the coastal houses are **not** prepared **for** heavy rainfall

Resultado: coincide con la secuencia objetivo de referencia.

=====

==

RESUMEN DEL PROCESO

=====

==

4 Epílogo

Llegamos al final de esta bonita y desafiante intención de comprensión de los Transformers. Los Transformers son un gran aporte a la inteligencia artificial moderna. Esa capacidad de analizar las relaciones entre los elementos de una secuencia y aprender a decidir cuáles son realmente importantes, es una cosa alucinante. Su mecanismo de atención es tan genial que presenta nuevos desafíos, especialmente cuando las secuencias son muy largas y el costo computacional aumenta. Ya se escucha fuertemente de Mamba, los investigadores precursores del tema lo dicen textualmente:

“Mamba disfruta de la inferencia rápida (5× mayor rendimiento que Transformers) y escalado lineal en la longitud de la secuencia, y su rendimiento mejora en secuencias reales de hasta un millón de longitudes” (Gu & Dao, 2024).

Pero como tenemos Transformers “para rato”, porque no se trata de simplemente reemplazar Transformers por Mamba. Cada arquitectura tiene fortalezas y limitaciones, y es probable que el futuro combine diferentes mecanismos según la tarea. Y siempre sea necesario tener los fundamentos de Transformers.

Más que aprender una arquitectura específica, comprender estos modelos nos enseña algo importante: que la inteligencia artificial avanza y no se detiene y que tenemos darnos la oportunidad de comprender las tecnologías que utilizamos, mientras estemos en esta tierra.

Te deseo el mejor de los éxitos, “senior”.

5 Bibliografía

- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Networks* (arXiv:1406.2661). arXiv. <https://doi.org/10.48550/arXiv.1406.2661>
- Gu, A., & Dao, T. (2024). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces* (arXiv:2312.00752). arXiv. <https://doi.org/10.48550/arXiv.2312.00752>
- Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the Dimensionality of Data with Neural Networks. *Science*, 313(5786), 504-507. <https://doi.org/10.1126/science.1127647>
- HLE. (2026). *Humanity's Last Exam Benchmark Leaderboard*. Artificial Analysis. <https://artificialanalysis.ai/evaluations/humanitys-last-exam>
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), 84-90. <https://doi.org/10.1145/3065386>

Lightcast. (s. f.). *The Stanford AI Index 2026*. Lightcast. Recuperado 31 de agosto de 2026, de <https://lightcast.io/resources/research/stanford-ai-index-2026>

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4), 115-133. <https://doi.org/10.1007/BF02478259>

McKinsey. (2026). *Global management consulting | McKinsey & Company*. <https://www.mckinsey.com/>

Minsky, M., Papert, S., & Bottou, L. (2017). *Perceptrons: An introduction to computational geometry* (Reissue of the 1988 expanded edition). The MIT Press.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. <https://doi.org/10.1037/h0042519>

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>

Russell, S. J., & Norvig, P. (with Corchado Rodríguez, J. M., & Joyanes Aguilar, L.). (2011). *Inteligencia artificial: Un enfoque moderno* (2ª ed., reimp). Pearson Educación.

Standford. (2026). *Education | The 2026 AI Index Report | Stanford HAI*. <https://hai.stanford.edu/ai-index/2026-ai-index-report/education>

Trova, G. (2026). A comprehensive EU AI Act Summary [August 2026 update].

SIG. <https://www.softwareimprovementgroup.com/blog/eu-ai-act-summary/>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). *Attention Is All You Need* (arXiv:1706.03762).

arXiv. <https://doi.org/10.48550/arXiv.1706.03762>